

Capítulo 1

Introducción

El proyecto “*Software para la gestión de activos en instalaciones de cartelería digital*” dirigido por Nelson Adrián Martínez Melongo y asesorado por Carles Farré Tost, se desarrolla en el marco del trabajo de fin de grado del grado en Ingeniería Informática de la Facultad de Informática de Barcelona, Universitat Politècnica de Catalunya, con mención en Ingeniería del Software. El trabajo se realiza en la modalidad B (proyecto realizado en empresa con Convenio de Cooperación Educativa) más concretamente en la empresa Waapiti S.L. (en adelante, *Waapiti*).

El principal objetivo de este trabajo consiste en el desarrollo de un módulo de gestión de activos que amplíe la plataforma software de *Waapiti* y permita dar solución a una serie de problemas y necesidades de la empresa.

En este primer capítulo se presenta el contexto en el que se desarrolla el proyecto, se definen los conceptos que se utilizarán a lo largo del mismo y se expone el problema que se pretende resolver. Además, se describen los actores que intervienen en el proyecto y se expone la motivación que ha llevado a su realización.

1.1. Contextualización

La empresa *Waapiti* ofrece soluciones de cartelería digital [1], que significa el uso de pantallas, monitores o paneles LED para la visualización de contenidos de forma dinámica, sustituyendo así a los carteles tradicionales usados comúnmente en publicidad o en ámbitos informativos. El software de la empresa permite la gestión de contenidos multimedia en pantallas distribuidas y conectadas a la red, de forma que se pueden actualizar dichos contenidos remotamente.

Pese a que el principal cometido de la plataforma es la gestión de los contenidos, también permite controlar usuarios y clientes, así como las ubicaciones donde se encuentran las pantallas. Esta mención a las ubicaciones es importante, ya que es una de las funcionalidades que se pretende mejorar en el proyecto.

Un ejemplo de uso de la plataforma podría ser:

Una empresa de ropa con más de cien tiendas repartidas por España crea una serie de vídeos para su nueva campaña. La empresa quiere mostrarlos en las pantallas que hay en los escaparates y en la zona de cajas. Para ello, el usuario encargado entra en

la plataforma, sube los contenidos y especifica unos horarios y condiciones. De forma automática estos contenidos se reproducen en las cien tiendas dadas las condiciones y los horarios especificados.

Dentro de la mención en ingeniería del software, este proyecto se centra en tecnologías web, más concretamente se usará *React* [2] y *Nest* [3] para el código base y servicios cloud de *AWS* [4] para la infraestructura. Las tecnologías usadas en el proyecto son las mismas que se usan en la plataforma de *Waapiti* y se explicarán en profundidad más adelante.

Con tal de facilitar el entendimiento del proyecto, no se definirá todo el funcionamiento y las posibilidades de la plataforma, pero sí todas aquellas partes que estén relacionadas con el módulo de gestión de activos y que por tanto sea necesario conocer.

1.2. Conceptos

Existe una terminología específica relacionada con la cartelería digital y el modelo de negocio de *Waapiti* que se debe conocer para entender el proyecto. En este apartado se definen todos estos conceptos que aparecerán a lo largo del trabajo con fin de facilitar la lectura y evitar confusiones.

- **Player:** Un player es un dispositivo externo que permite reproducir contenidos en una pantalla no profesional ¹. Consiste en un mini ordenador que conectado a la pantalla permite enlazarse con la plataforma de gestión de contenidos y reproducir los diferentes materiales audiovisuales.
- **Location:** Una location es un punto geográfico o ubicación en el que se encuentra una instalación de cartelería digital. Por ejemplo, una tienda, un centro comercial, una estación de tren, etc.
- **Activo:** Un activo es todo elemento físico que se encuentra en una instalación de cartelería digital y que permite, conectado con los otros activos, el objetivo final de reproducir contenidos en pantallas. Como ejemplo, un activo puede ser la propia pantalla, un commutator de red [6], un player, etc.
- **Instalación:** Una instalación es la relación entre un conjunto de activos y una location. Permite definir los detalles y diferentes datos relacionados con el montaje de los activos. Es importante entender que en muchos casos hablar de instalación es sinónimo de hablar de location para el contexto de este proyecto.
- **Intervención:** Una intervención es un evento que se produce cuando un instalador o técnico debe asistir personalmente a una instalación para realizar una tarea. Por ejemplo, cambiar una pantalla, instalar un player, etc. **La instalación inicial se considera como la primera intervención de la location.**
- **SAT:** SAT o Servicio de Atención Técnica es el departamento de *Waapiti* que se encarga del control y el soporte de las instalaciones. Es el encargado de gestionar las incidencias que se producen y dar la respuesta adecuada a los clientes.

¹Las pantallas profesionales o pantallas SOC [5] tienen un player integrado en su interior.

- **Partners:** Los partners son empresas que ya disponen de sus clientes pero necesitan de la plataforma de *Waapiti* para ofrecer un mejor servicio o una vía de negocio alternativa. Por ejemplo, una empresa de publicidad que ya tiene clientes pero que necesita de una plataforma para gestionar la cartelería digital. Los partners suponen la mayor parte del modelo de negocio de *Waapiti*.

1.3. Descripción del problema

En este proyecto se van a solucionar dos problemas que se presentan en la empresa actualmente, ambos relacionados.

- **Desunión en el proceso de instalación:** El SAT no tiene una herramienta que le permita gestionar las instalaciones de forma eficiente. Actualmente la información de las instalaciones se encuentra dispersa en diferentes documentos y departamentos y no existe una herramienta que permita gestionarlas de forma centralizada. Esto genera una pérdida de tiempo causada por tareas duplicadas y por otro lado una pérdida de información que puede ser útil a posteriori para el propio SAT o incluso para los clientes.
- **Dificultades en el soporte:** Una de las funciones del SAT es dar soporte a los clientes. Muchas veces resulta complicado vía telefónica conseguir identificar algunos de los activos que fallan y por tanto necesitan de una serie de pruebas para identificar el posible error. Esto se debe a que por una parte el cliente normalmente no tiene un perfil técnico, y por otro lado el SAT no puede recordar exactamente el aspecto de cada modelo de cada uno de los activos que se encuentran en la instalación.

Para ejemplificar algunas de las dificultades, se detallan a continuación el caso habitual en una instalación y el soporte una vez en funcionamiento:

1. Se recogen los requisitos del cliente y se envía al departamento de compras una lista con los activos necesarios, tales como pantallas, cables, routers, etc.
2. Tras cerrar el presupuesto el SAT estudia la instalación y especifica los detalles en un mail para el instalador externo ².
3. El mismo SAT genera un formulario especificando de nuevo los activos de forma que el instalador puede validar su trabajo y añadir fotografías. (Esta información se utiliza para validar el trabajo del instalador únicamente, por tanto se pierde)
4. En un momento dado, uno de los activos empieza a fallar por algún motivo. El SAT llama al cliente y mediante descripciones genéricas debe conseguir que el cliente identifique el activo que se quiere probar.
5. En caso que se tenga que sustituir, el SAT tendrá que preguntar al departamento de compras si el activo sigue en garantía ³. Si finalmente se sustituye, este cambio no se refleja en ningún documento de fácil acceso y por tanto no se puede saber de forma sencilla en el futuro.

²El montaje de los activos se delega a instaladores ajenos a *Waapiti*

³Por ley se debe ofrecer dos años de garantía para cada activo tecnológico

Resumidamente, el proceso es ineficiente, se pierde información y se desperdicia tiempo. Por tanto, se debe buscar una solución que permita mejorar el proceso de instalación y el soporte.

1.4. Actores implicados

En esta sección se describen los actores que intervienen en la creación del proyecto, así como aquellos que se verán beneficiados o afectados por el mismo.

SAT: Previamente definido (1.2), el SAT es el principal beneficiario de este proyecto y por tanto es el actor que define la mayoría de requisitos. Al ser un actor interno, tendrá un papel activo en la elaboración del proyecto, participando en algunas reuniones y aportando feedback.

Departamento de compras: También se verá afectado por este TFG, aunque en menor medida, no tendrá un papel activo durante el desarrollo, pero sí que define algunos requisitos iniciales.

Product Owner: Es el encargado de maximizar el valor del producto y asegurar un retorno de la inversión. Representa a los clientes dentro del equipo y a su vez es la cara visible del proyecto ante los clientes. Es el encargado de decidir qué funcionalidades se implementarán en cada sprint, aunque esta será una responsabilidad compartida con el desarrollador del proyecto.

Desarrollador: Es la persona que diseñará e implementará todo el software, documentará el proyecto y se encargará de recoger los requisitos de los diferentes actores. Cabe destacar que este desarrollador es el autor de este TFG y será quien realice desde el diseño UX/UI [7] hasta la implementación de backend y frontend [8].

Partners: De la misma forma que el SAT, este proyecto también beneficiará a los partners. Podrán utilizar el nuevo módulo de la plataforma para mejorar sus servicios y su eficiencia.

Director del proyecto: El director del proyecto y programador senior de *Waapiti* supervisará la corrección en la implementación del código y propondrá mejoras en el diseño de la solución.

1.5. Motivación

Previamente a la selección del tema de este TFG, se realizó una búsqueda de potenciales proyectos que fuesen beneficiosos para *Waapiti* y que a su vez fueran interesantes para el autor del TFG. Los requisitos para la selección eran los siguientes:

- El proyecto debía ser un módulo completo, es decir, tenía que incluir todos los pasos habituales en el desarrollo de un software: análisis, diseño, implementación, pruebas y documentación.
- El proyecto debía incluir componentes de las diferentes disciplinas del desarrollo web, en especial de backend, puesto que el autor no tenía experiencia en este campo

y quería aprender más sobre él.

- El proyecto debía partir de una necesidad actual, con tal de asegurar la recompensa de la inversión de tiempo y económica.
- El proyecto no podía tener una complejidad excesiva, puesto que debe limitarse al tiempo disponible para su realización y a las capacidades reales de aprendizaje del autor.

Tras valorar distintas opciones, se llegó a la conclusión de que el proyecto de gestión de activos en instalaciones de cartelería digital era el más adecuado para el autor y para *Waapiti*. Este proyecto cumple con todos los requisitos anteriores y supone para el autor un gran reto alcanzable, que le permitirá aprender y desarrollar nuevas habilidades.

Capítulo 7

Especificación de requisitos

En este capítulo se detallan los requisitos que debe cumplir el sistema. A diferencia de la sección 3.2, se detallan en profundidad las condiciones de satisfacción de cada requisito.

7.1. Requisitos funcionales

Los requisitos funcionales son aquellos que definen las funcionalidades del sistema. Para entenderlos, se han definido historias de usuario que describen las funcionalidades desde el punto de vista del usuario.

7.1.1. Historias de usuario

Una historia de usuario es una explicación general e informal de una función de software pensada y escrita desde el punto de vista del usuario final. Su propósito es articular cómo estas funcionalidades aportaran un valor particular al cliente. Las historias de usuario siguen el siguiente patrón:

Como [Rol], quiero poder [Acción] para [Beneficio]

Donde los elementos entre corchetes se corresponden con:

- **Rol:** El usuario que quiere realizar la acción. Podría ser un usuario final, un administrador, un cliente, etc.
- **Acción:** La acción que el usuario quiere realizar. Por ejemplo, crear una cuenta, iniciar sesión, etc.
- **Beneficio:** El beneficio que el usuario obtiene al realizar la acción. Por ejemplo, visualizar sus datos, recibir notificaciones, etc.

Para cada historia de usuario se ha definido su Acceptance Criteria (criterios de aceptación), que es un conjunto de condiciones que deben ser cumplidas para que la historia de usuario se considere finalizada.

Gestión de locations (instalaciones)

Historia de usuario	H1: Crear spots
Como	Usuario de SAT
Quiero poder	Crear nuevos spots en una location
Para	Poder añadir los puntos en los que se distribuyen los activos
Criterios de aceptación	▪ Se puede asignar un nombre a cada spot ▪ Se asigna automáticamente la planta o se pueden crear nuevas ▪ Se ve el spot creado automáticamente

Tabla 7.1: Historia de usuario 1

Historia de usuario	H2: Eliminar spots
Como	Usuario de SAT
Quiero poder	Eliminar spots de una location
Para	Poder modificar la distribución de una location
Criterios de aceptación	▪ No se puede eliminar un spot si tiene activos asignados ▪ Se elimina el spot de la lista automáticamente

Tabla 7.2: Historia de usuario 2

Historia de usuario	H3: Crear activos en spot
Como	Usuario de SAT
Quiero poder	Crear activos asignados a un spot
Para	Poder tener el control de los activos y sus características
Criterios de aceptación	▪ Se debe poder añadir al activo todos los atributos de activo y de tipo de activo ▪ Se asigna automáticamente el spot, aunque se puede decidir en qué spot se quiere asignar ▪ Se puede dejar en blanco el número de serie, en cuyo caso se asignará en la instalación

Tabla 7.3: Historia de usuario 3

Historia de usuario	H4: Eliminar activos de spot
Como	Usuario de SAT
Quiero poder	Eliminar activos de un spot
Para	Poder rectificar en errores de creación
Criterios de aceptación	▪ Se elimina el activo de la lista automáticamente ▪ Solo se puede eliminar un activo si no está asignado a una intervención

Tabla 7.4: Historia de usuario 4

Historia de usuario	H5: Modificar información de activo
Como	Usuario de SAT
Quiero poder	Modificar la información de un activo
Para	Poder rectificar en errores de creación
Criterios de aceptación	▪ Se puede modificar cualquier atributo del activo ▪ Se puede modificar el spot al que está asignado

Tabla 7.5: Historia de usuario 5

Historia de usuario	H6: Introducir ID de mapa 3D
Como	Usuario de SAT
Quiero poder	Introducir el ID del mapa 3D
Para	Poder visualizar el mapa 3D de la location
Criterios de aceptación	▪ No se puede introducir un nuevo ID una vez asignado

Tabla 7.6: Historia de usuario 6

Visualización de activos

Historia de usuario	H7: Ver activos de location
Como	Cualquier tipo de usuario
Quiero poder	Ver los activos que hay distribuidos en una location
Para	Poder entender la distribución de activos o dar soporte (en caso de ser usuario SAT)
Criterios de aceptación	▪ No se pueden mostrar activos fuera de la location ▪ Se debe poder ver los detalles de cada activo

Tabla 7.7: Historia de usuario 7

Historia de usuario	H8: Ver activos en forma de mapa 3D
Como	Cualquier tipo de usuario
Quiero poder	Ver los activos que hay distribuidos en una location en forma de mapa 3D
Para	Poder entender la distribución de activos o dar soporte (en caso de ser usuario SAT)
Criterios de aceptación	▪ No se pueden mostrar activos fuera de la location ▪ Se debe poder ver los detalles de cada activo ▪ La información debe ser la misma que en la vista de lista

Tabla 7.8: Historia de usuario 8

Historia de usuario	H9: Ver todos los activos en una tabla
Como	Cualquier tipo de usuario
Quiero poder	Ver todos los activos de los que dispongo
Para	Poder controlar los activos que tengo y su estado
Criterios de aceptación	▪ Ningún usuario puede ver activos de los que no disponga (menos administradores, que ven todos) ▪ Se debe poder ver los detalles de cada activo

Tabla 7.9: Historia de usuario 9

Gestión de intervenciones

Historia de usuario	H10: Crear nueva intervención
Como	Usuario de SAT
Quiero poder	Crear una nueva intervención
Para	Poder generar una acción real donde se intervenga uno o varios activos (o la location)
Criterios de aceptación	▪ Se debe poder escoger los activos intervenidos ▪ Cada activo se asociará a una acción ▪ No se deben poder incluir activos que no están en la location o que ya están en una intervención

Tabla 7.10: Historia de usuario 10

Historia de usuario	H11: Eliminar una intervención
Como	Usuario de SAT
Quiero poder	Eliminar una intervención
Para	Poder rectificar en errores de creación
Criterios de aceptación	▪ Se elimina la intervención de la lista automáticamente ▪ Solo se puede eliminar una intervención si no está finalizada

Tabla 7.11: Historia de usuario 11

Historia de usuario	H12: Completar una intervención
Como	Técnico
Quiero poder	Realizar y completar una intervención
Para	Poder dar por finalizada una intervención y completar los datos necesarios
Criterios de aceptación	▪ Se debe ver toda la información de la intervención de forma sencilla ▪ Para poder finalizar la intervención se tendrá que rellenar toda la información exigida

Tabla 7.12: Historia de usuario 12

Historia de usuario	H13: Visualizar registro de intervenciones
Como	Cualquier tipo de usuario
Quiero poder	Ver el historial de intervenciones realizadas
Para	Poder mantener un control de todo lo que ha pasado en una location o sobre un activo
Criterios de aceptación	▪ Debe poderse consultar el registro a nivel de location o a nivel de activo

Tabla 7.13: Historia de usuario 13

7.2. Requisitos no funcionales

En esta sección se describen los requisitos no funcionales del sistema, estos requisitos son los que describen las características que debe tener el sistema, pero no son parte de la funcionalidad del mismo.

Para definir cada requisito con exactitud, se describirán acompañado de una justificación del porqué es necesario y una criterio de satisfacción que permita verificar que el requisito se cumple.

Estos requisitos no funcionales siguen la plantilla de *Volere Requirements Specification* [16]

Requisito	10a. Apariencia
Descripción	La interfaz debe seguir el estilo y los colores de la plataforma <i>Waapiti</i> .
Justificación	La empresa <i>Waapiti</i> tiene una imagen corporativa que se debe mantener en todos sus productos con tal de generar confianza y valor en los clientes.
Aceptación	Todos los colores usados en la interfaz se corresponden con los definidos en la paleta de color de <i>Waapiti</i> incluida en el código de la plataforma.

Tabla 7.14: Requisito no funcional 1

Requisito	11b. Personalización e internacionalización
Descripción	La interfaz debe poderse cambiar a catalán, español, inglés, francés, alemán, italiano y portugués.
Justificación	La plataforma <i>Waapiti</i> se usa en varios países y por tanto se debe poder adaptar a los distintos idiomas que se hablan.
Aceptación	La interfaz se puede entender por cualquier persona que hable uno de los 7 idiomas y el idioma por defecto es el inglés.

Tabla 7.15: Requisito no funcional 2

Requisito	11c. Aprendizaje
Descripción	El sistema debe ser fácil de aprender para los usuarios técnicos.
Justificación	El principal usuario de este proyecto son usuarios técnicos que deben dominar todas las funcionalidades sin dedicar un tiempo excesivo a su aprendizaje.
Aceptación	Los usuarios técnicos aprenden a usar las funcionalidades de la plataforma en menos de 30 minutos.

Tabla 7.16: Requisito no funcional 3

Requisito	12a. Velocidad y latencia
Descripción	El sistema debe responder rápidamente ante acciones del usuario.
Justificación	El usuario no puede esperar demasiado ante acciones de la plataforma ya que esto puede generar frustración y desconfianza en el sistema.
Aceptación	El sistema responde a cualquier acción del usuario en menos de cinco segundos.

Tabla 7.17: Requisito no funcional 4

Requisito	12d. Disponibilidad y fiabilidad
Descripción	El sistema debe estar siempre disponible, excepto en casos de actualización programada.
Justificación	La plataforma <i>Waapiti</i> se usa las 24 horas del día y por tanto no se puede permitir que el sistema no esté disponible.
Aceptación	Cualquier usuario puede acceder a la plataforma en cualquier momento del día. A excepción de actualizaciones programadas.

Tabla 7.18: Requisito no funcional 5

Requisito	12f. Capacidad
Descripción	El sistema debe soportar un número ilimitado de conexiones simultáneas.
Justificación	El sistema tiene un gran número de usuarios y todos deben poder acceder a él, sin importar cuánta gente esté conectada.
Aceptación	La plataforma escala de forma dinámica con los cambios de conexiones simultáneas.

Tabla 7.19: Requisito no funcional 6

Requisito	14c. Adaptabilidad
Descripción	El sistema debe adaptarse a cambios de pantalla, por ejemplo de un ordenador de sobremesa a un móvil.
Justificación	La plataforma <i>Waapiti</i> se usa en distintos dispositivos y por tanto se debe poder utilizar independientemente del tamaño de pantalla.
Aceptación	La interfaz se adapta a pantalla de ordenador, tableta y móvil.

Tabla 7.20: Requisito no funcional 7

Requisito	15a. Acceso
Descripción	Las funcionalidades del proyecto deben ser accesibles solo para los usuarios autorizados.
Justificación	Cada cliente de <i>Waapiti</i> contrata ciertas funcionalidades y no se debe permitir el acceso en caso de que intenten acceder a funcionalidades que no tienen.
Aceptación	Los usuarios sin permiso no pueden acceder a ver y editar activos.

Tabla 7.21: Requisito no funcional 8

Requisito	15c. Privacidad
Descripción	La información confidencial de los usuarios debe estar protegida y se debe garantizar su privacidad.
Justificación	La plataforma <i>Waapiti</i> almacena información confidencial de los usuarios y por tanto se debe garantizar su privacidad.
Aceptación	Ningún usuario puede ver información más allá de la suya propia.

Tabla 7.22: Requisito no funcional 9

7.3. Modelo conceptual del dominio

En la figura 7.1 se puede ver el diagrama conceptual del dominio que describe este proyecto.

Se debe destacar Location, Intervention y Asset, ya que son el principal motivo de valor de la herramienta. Por supuesto, las otras clases especificadas son necesarias para el correcto funcionamiento del gestor de activos.

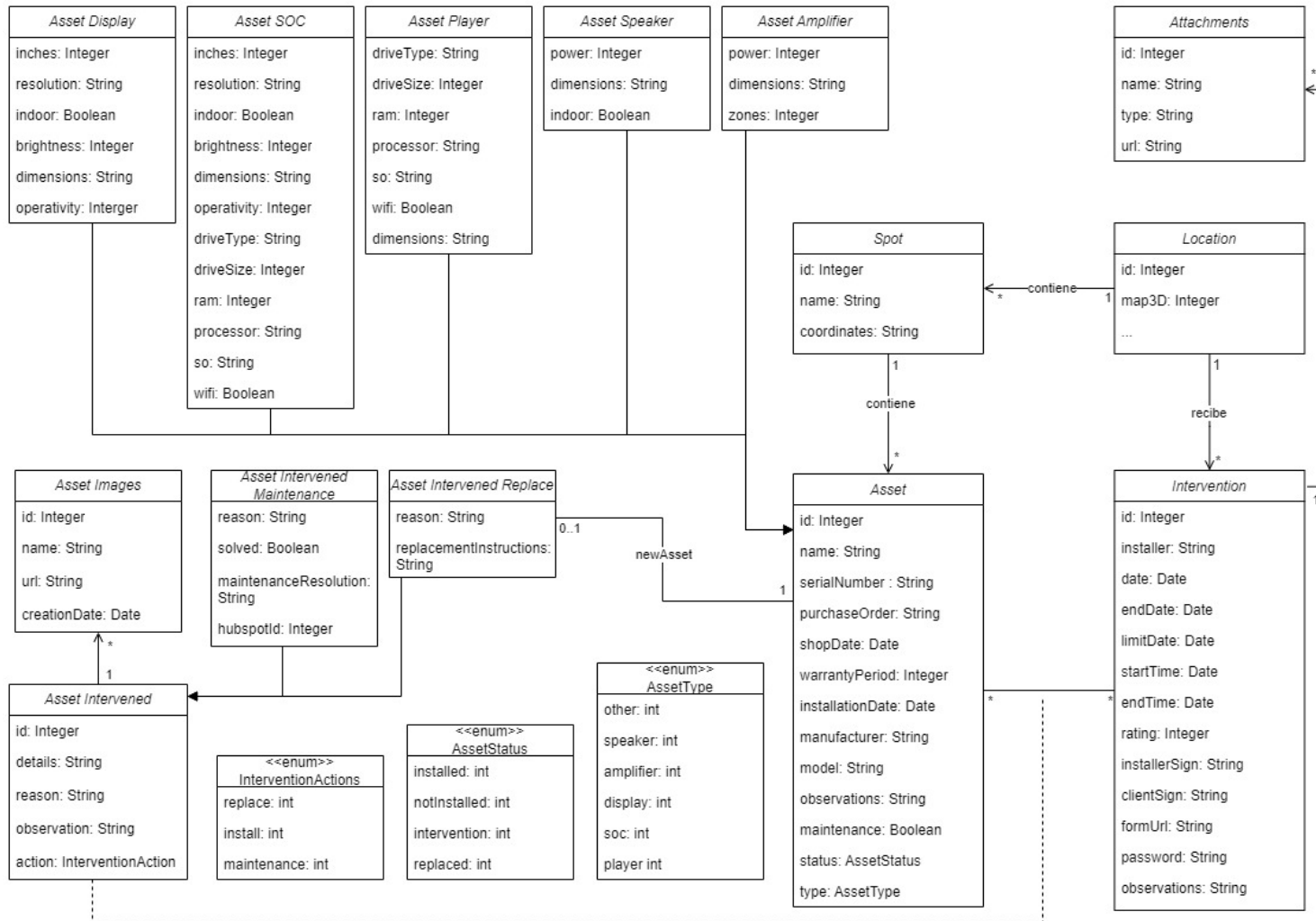


Figura 7.1: Diagrama conceptual del dominio

Fuente: Elaboración propia

Capítulo 8

Arquitectura

En este capítulo se describe la arquitectura del sistema. Se ha dividido en dos secciones, la primera de ellas describe la arquitectura lógica, y la segunda la arquitectura física.

8.1. Arquitectura lógica

Arquitectura Cliente-Servidor

La plataforma de *Waapiti* es un servicio web, por tanto, como es habitual en este tipo de softwares, sigue la arquitectura cliente-servidor.

Por un lado está el cliente, quien realiza las diferentes peticiones, y por otro lado está el servidor, quien las recibe y las procesa. En este caso, el cliente es el usuario de la plataforma, y el servidor es el software que se encarga de procesar las peticiones del usuario.

La comunicación entre ambos se realiza a través de internet utilizando el modelo de REST API [30] y el protocolo de comunicación HTTP [31].

Arquitectura de 3 capas

Para desacoplar las partes que componen el modelo de cliente-servidor, se ha utilizado la arquitectura de 3 capas. Esta arquitectura divide el sistema en tres partes, la capa de presentación, la capa de negocio y la capa de datos.

- **Capa de presentación:** Es la capa que se encarga de mostrar la información al usuario. En este caso, la capa de presentación es el cliente, es decir, el navegador web del usuario.
- **Capa de negocio:** Es la capa que se encarga de procesar la información. En este caso, la capa de negocio es el servidor, es decir, el software que se encarga de procesar las peticiones del usuario.
- **Capa de datos:** Es la capa que se encarga de almacenar la información.

8.1.1. Arquitectura lógica de frontend

Para el desarrollo de la capa de presentación se ha utilizado una arquitectura lógica llamada Flux.

Flux

Flux es una arquitectura de frontend desarrollada por Facebook para el desarrollo de aplicaciones web. Esta arquitectura se basa en el patrón de diseño Modelo-Vista-Controlador (MVC) [32], pero con la diferencia de que no existe una comunicación bidireccional entre la vista y el modelo, sino que la comunicación es unidireccional.

En el frontend de la aplicación se ha utilizado el framework React, el cuál se detalla en el capítulo de tecnologías 9. Junto con la librería Redux [33] se puede implementar la arquitectura Flux.

Esta arquitectura se compone de cuatro partes:

- **Vista:** Es la parte encargada de mostrar la información al usuario. En este caso, la vista se corresponde a los componentes de React.
- **Acciones:** Son las acciones que puede realizar el usuario. Se definen como objeto de JavaScript e indican la intención de modificar el estado de la aplicación.
- **Dispatcher:** Es el encargado de recibir las acciones y enviarlas al store para que modifique su estado.
- **Store:** Es el encargado de almacenar el estado de la aplicación. Cuando recibe una acción por parte del dispatcher, modifica su estado y notifica a todas las vistas asociadas para que se actualicen.

En la figura 8.1 se puede ver un esquema de la arquitectura Flux.

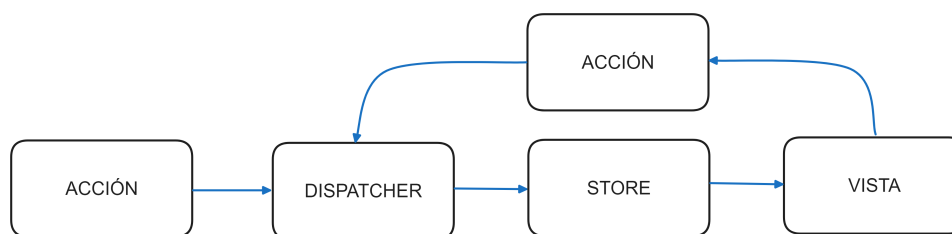


Figura 8.1: Arquitectura Flux

Fuente: Elaboración propia

Redux

Redux es una librería de JavaScript que permite implementar la arquitectura Flux. La principal diferencia con Flux es que en Redux no existe el concepto de Dispatcher, sino que las acciones se envían directamente al store. Redux se basa en tres principios:

- **Única fuente de la verdad:** El estado de la aplicación se almacena en un único objeto llamado store.
- **Solo lectura:** El estado de la aplicación solo se puede modificar mediante acciones.

- **Cambios mediante funciones puras:** Las funciones que modifican el estado de la aplicación se denominan reducers y son funciones puras. Esto quiere decir que no modifican el estado de la aplicación, sino que devuelven un nuevo estado.

Se ha escogido Redux por su facilidad de integración con React y por su gran comunidad de desarrolladores.

En la figura 8.2 se puede ver un esquema de la arquitectura Redux en el uso particular de la plataforma de *Waaipiti*.

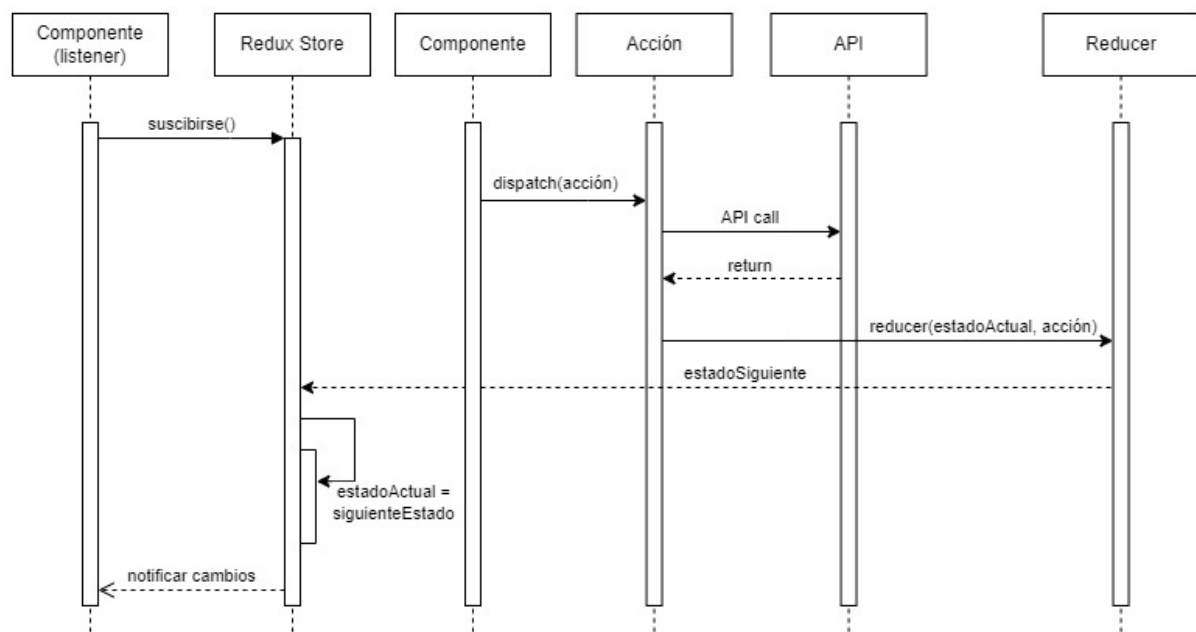


Figura 8.2: Diagrama de secuencia Redux

Fuente: Elaboración propia

8.2. Arquitectura física

En el caso de la plataforma de *Waaipiti*, la arquitectura física no se corresponde con la arquitectura lógica. Tanto la capa de negocio como la capa de datos se dividen en diferentes servicios (cada uno ejecutado en máquinas diferentes) para mejorar la seguridad, la escalabilidad y la disponibilidad del sistema.

Capa de presentación

En este caso, la capa de presentación no tiene complejidad. Esta se corresponde con el navegador web del usuario.

Capa de negocio

La capa de negocio está dividida en dos servicios alojados en diferentes máquinas. De forma interna en el equipo de desarrollo se les asigna el nombre de *business* y *functional*.

Esta separación permite añadir seguridad al sistema ya que el único acceso público es al servicio *business*, que tan solo contiene la información mínima para entender las peticiones del usuario. Estas peticiones son procesadas y enviadas mediante una conexión

interna al servicio *functional*, que es el que contiene la información sensible del sistema. La comunicación se realiza mediante el protocolo de comunicación TCP [34].

- **Servicio *business*:** Se encarga de la gestión de usuarios, la autenticación y la autorización de los mismos. Define las rutas de la API REST y procesa las peticiones para enviarlas al servicio *functional*.
- **Servicio *functional*:** Este es el servicio que realiza toda la lógica importante de la plataforma. Es el servicio que tiene comunicación con la capa de datos.

Capa de datos

La capa de datos se divide en varios servicios de AWS que permiten mejorar el rendimiento y la escalabilidad del sistema.

- **RDS:** RDS es un servicio de base de datos relacional distribuido. En el caso de *Waapiti* se utiliza una base de datos MySQL y contiene toda la información del sistema.
- **Amazon S3:** Es el servicio de almacenamiento de objetos de AWS. Se utiliza para almacenar los archivos multimedia.
- **Cache de usuarios:** Para mejorar la velocidad de respuesta del sistema, existe un sistema de caché de usuarios almacenada en Amazon DynamoDB, a la cuál tiene acceso el servicio *business*.

En la figura 8.3 se puede ver un esquema de la arquitectura del sistema.

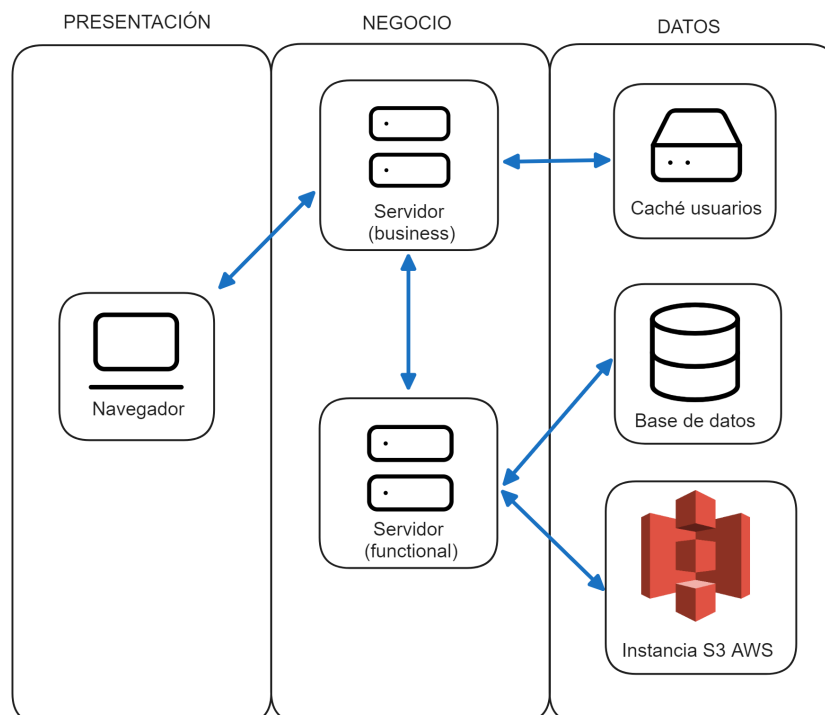


Figura 8.3: Arquitectura del sistema

Fuente: Elaboración propia

8.3. Patrones de diseño

Durante el desarrollo del TFG se han utilizado varios patrones de diseño. En esta sección se explican los más importantes.

8.3.1. Patrón observador

El patrón observador es un patrón de diseño de software que define una dependencia de uno a muchos entre objetos, de forma que cuando un objeto cambia su estado, todos los objetos que dependen de él son notificados y actualizados automáticamente.

Este patrón se encuentra en la arquitectura Flux, ya que el store notifica a las vistas cuando cambia su estado.

8.3.2. Patrón provider

El patrón provider es un patrón de diseño de software que permite compartir información entre componentes de React sin necesidad de utilizar *props*.

Este patrón se utiliza en la plataforma para compartir la información de localización entre los componentes de tipo fecha. De esta forma se puede mostrar la fecha en el formato horario de la localización seleccionada, y ante un cambio de localización, todos los componentes de tipo fecha se actualizan automáticamente.

8.3.3. Patrón módulo

Este es el patrón más importante dentro del lenguaje de JavaScript. Este patrón permite encapsular la información y exponer solo aquella que se desea compartir entre módulos (como funciones o componentes).

8.3.4. Patrón gancho (hook)

Los ganchos son una nueva característica de React que permiten utilizar el estado y otras características de React sin necesidad de escribir una clase. Los ganchos son funciones que permiten ^aengancharse al estado de React. Se utilizan en este proyecto principalmente para controlar el estado de los formularios.

Estos *hooks* se optimizan automáticamente por React y su uso es recomendable para extraer la lógica de los componentes.

Capítulo 9

Tecnologías

En este capítulo se describen las tecnologías y *frameworks* [35] utilizados para el desarrollo del código del proyecto.

Todas las tecnologías contribuyen muy favorablemente al aprendizaje del autor del TFG puesto que son algunas de las tecnologías más utilizadas a día de hoy en el desarrollo de aplicaciones web.

9.1. Tecnologías y frameworks utilizados

- **React:** [2] es una librería (considerada framework) de JavaScript para construir interfaces de usuario. Se utiliza para crear componentes reutilizables y dinámicos. Es el principal framework utilizado en el frontend i sin su uso sería muy complicado poder crear una aplicación web como la que se ha desarrollado. No se han considerado alternativas a React como *Angular* [36] o *Vue* [37] ya que React es el framework más utilizado en la actualidad y es el que se decidió usar para la plataforma *Waapiti* antes del inicio del TFG.
- **NestJS:** [3] es un framework de NodeJS para crear aplicaciones backend. Se ha utilizado para crear la API REST i sin su uso no se podría crear una API REST de forma tan segura y compleja pero a su vez fácil de trabajar. Se ha decidido utilizar NestJS en vez de alternativas populares como *Express* [38] por su estructura opinionada que permite crear APIs REST complejas.
- **TypeScript:** [39] es un superconjunto de JavaScript. Se ha utilizado para desarrollar tanto el backend. Se ha decidido utilizar TypeScript en vez de JavaScript para backend por su tipado estático que permite detectar errores en tiempo de compilación y debido a que es el lenguaje por defecto de NestJS. Para frontend se ha utilizado JavaScript.
- **TypeORM:** [40] es un ORM (Object Relational Mapping) [41] para TypeScript y JavaScript. Se ha utilizado para crear las entidades de la base de datos y para realizar las consultas a la base de datos. Se ha decidido utilizar TypeORM en vez de acceder directamente mediante SQL por su facilidad de uso y su integración con NestJS.

- **Material UI** [42] es una librería de React que contiene componentes de la interfaz de usuario. Permite agilizar la creación de interfaces y es una de las más utilizadas.
- **Infraestructura AWS** [4] es un conjunto de servicios de computación en la nube que permite crear aplicaciones escalables y de alta disponibilidad. Se ha utilizado para desplegar la aplicación web y la API REST. Se ha decidido utilizar AWS en vez de otras alternativas como *Google Cloud* [43] o *Microsoft Azure* [44] por su popularidad y su bajo coste.

9.2. Tecnologías y frameworks alternativos

Tras haber finalizado el desarrollo del proyecto, hay algunas tecnologías que se han quedado fuera del alcance del proyecto pero que se podrían utilizar en un futuro para mejorar la aplicación.

- **NextJS:** [45] es un meta framework de React que permite crear aplicaciones web de una forma sencilla (ya que facilita algunas utilidades como el enrutado de la web). Además, permite ejecutar código de servidor, favoreciendo la velocidad de la interfaz y por tanto mejorando la experiencia de usuario.
- **Shadcn:** [46] es un proyecto de componentes reusables de UI que dispone de un diseño moderno y que pone el foco en la accesibilidad y la facilidad para personalizar los componentes.

Capítulo 10

Diseño

En este capítulo se muestra y se explica el proceso que se ha seguido para crear los prototipos de la aplicación web.

Todos los prototipos se han creado utilizando la herramienta *Figma* y permiten pensar y analizar de forma anticipada la interfaz de usuario. Es necesario saber que la información de los prototipos no necesariamente corresponde con exactitud a la información que se muestra en la aplicación final.

En varias interfaces, compartir el diseño con otros miembros de la organización, ha permitido recibir *feedback* y mejorar los prototipos. Las pruebas de usabilidad se definen en la sección 12.1

10.1. Prototipos

Lista de activos

La lista de activos permite visualizar todos los activos que se han creado en la plataforma. Gracias a los filtros y a la búsqueda, se pueden encontrar activos de forma rápida. En la figura 10.1 se muestra el prototipo de la lista de activos.

The screenshot shows the 'Assets' page in the WAAPITI system. On the left is a sidebar with the UPCT logo and navigation links: Dashboard, Media Contents, Channels, Locations & Players, Assets (selected), Schedules, Users, and Settings. The main area displays a table of assets with columns: NAME, LOCATION, TYPE, WARRANTY, SERIAL NUMBER, and SHOP DATE. The table contains five rows of data, with the first row highlighted. A search bar and a 'Rows per page' dropdown (set to 50) are also visible.

NAME ↓	LOCATION	TYPE	WARRANTY	SERIAL NUMBER	SHOP DATE ↓
☒ Samsung TV	Alcala 2	Screen	Expired	0001249812	Jan 02, 2018
☐ 4 Inputs	Montmany 18	Switch	Jun 30, 2024	4320258623	Jun 30, 2022
☐ Main Player Pending Intervention	Carles August 34	Player	May 12, 2025	75466789654	May 12, 2023
☐ Alargo 8	San Agustin, 9	Other			May 11, 2023

Figura 10.1: Lista de activos

Fuente: Elaboración propia

Información de activo

La interfaz de información de activo permite visualizar todos los datos de un activo. En la figura 10.2 se muestra el prototipo de la información de activo.

The screenshot shows the 'Samsung TV' asset detail page. The sidebar is identical to the previous figure. The main area has a breadcrumb 'ASSETS' and a back arrow. Below the asset name 'Samsung TV' is an 'EDIT' button. The page is divided into two tabs: 'INFO' (selected) and 'HISTORY'. The 'INFO' tab contains two columns of data: 'General Information' and 'Installation'. Below the text is a photo of the Samsung TV.

General Information	Installation
Name Samsung TV Serial Number 0001249812 Type Screen Shop Date Jan 02, 2018 Warranty Expiration Expired - Jan 02, 2020	Location Alcala 2 Client UPC Installation Date Jan 10, 2018 Billing Date Jan 03, 2018 Client Warranty Expiration Expired - Jan 03, 2020 Installer Manuel Santos Tresvilla

Figura 10.2: Información de activo

Fuente: Elaboración propia

Historial de intervenciones de activo

El historial de intervenciones de activo permite visualizar todas las intervenciones que se han realizado sobre un activo, en el desarrollo se ha cambiado el nombre de la pestaña a intervenciones”. En la figura 10.3 se muestra el prototipo del historial de intervenciones de activo.

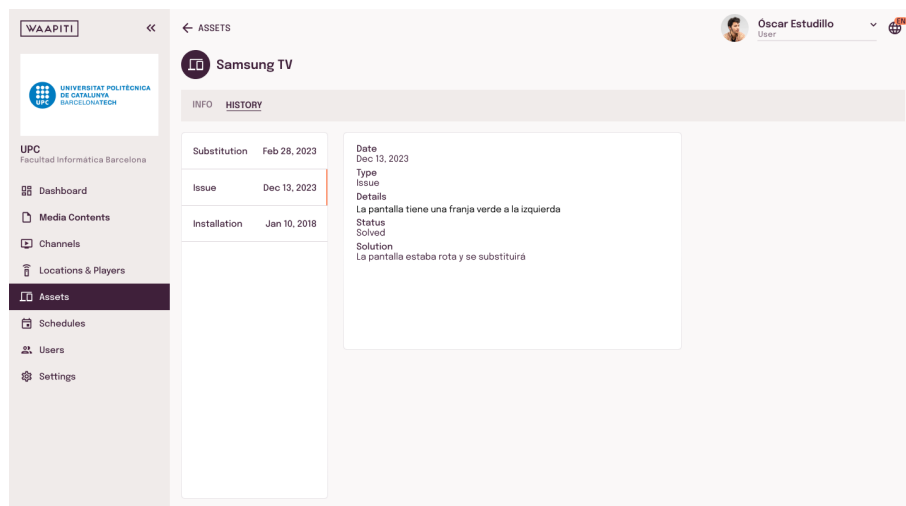


Figura 10.3: Historial de intervenciones de activo

Fuente: Elaboración propia

Información de location / instalación

La vista de location permite visualizar toda la información de una location, así como los assets que están instalados. En la figura 10.4 se muestra el prototipo de la información de location.

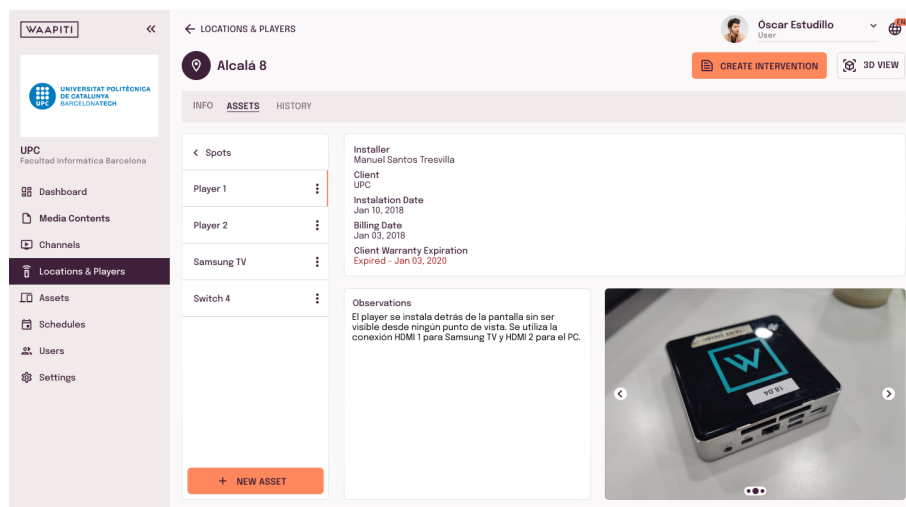


Figura 10.4: Información de location

Fuente: Elaboración propia

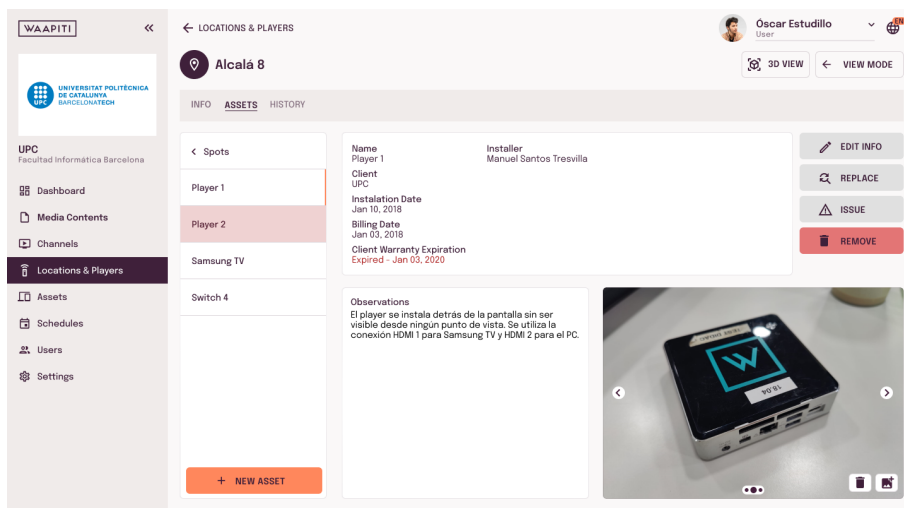


Figura 10.5: Información de location previo a test de usabilidad

Fuente: Elaboración propia

Información de location con mapa 3D

Como se ha explicado previamente, una de las formas de visualizar los activos instalados es mediante un mapa 3D. En la figura 10.6 se muestra el prototipo de la información de location con el mapa 3D.

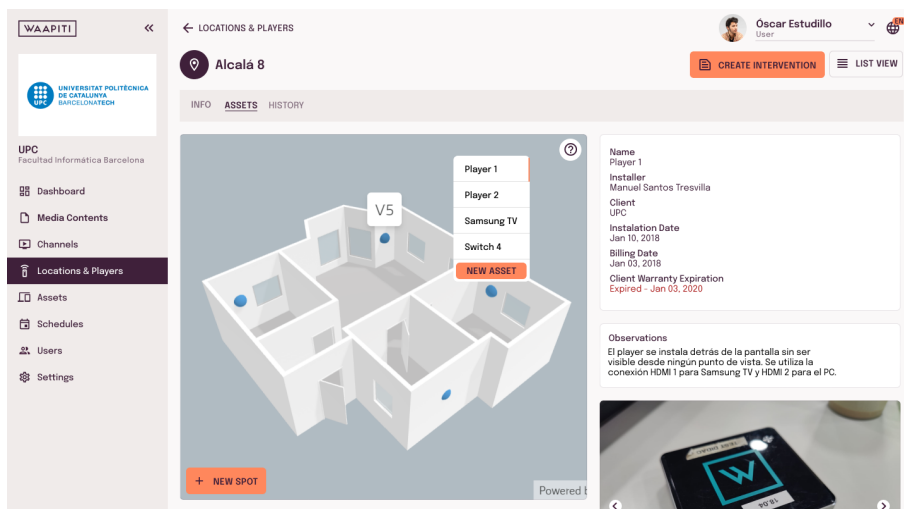


Figura 10.6: Información de location con mapa 3D

Fuente: Elaboración propia

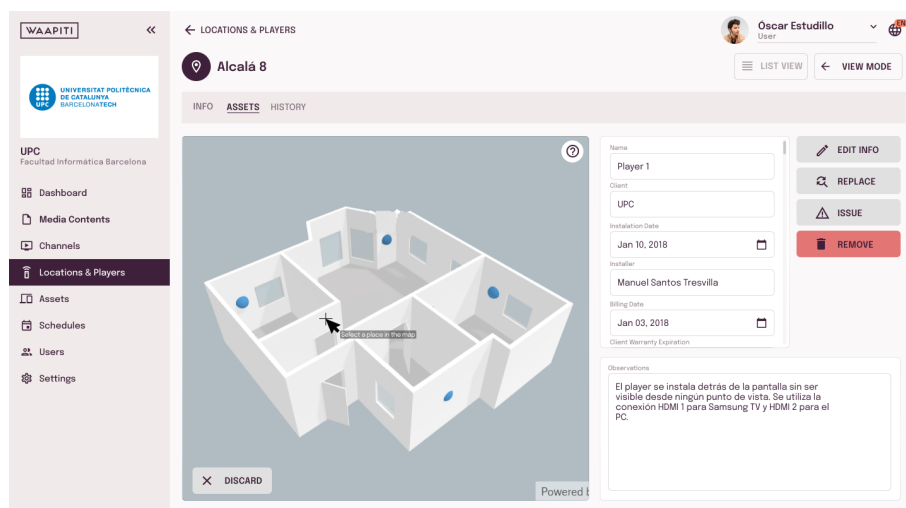


Figura 10.7: Información de location con mapa 3D previo a test de usabilidad

Fuente: Elaboración propia

Historial de intervenciones de location

El historial de intervenciones de location permite visualizar todas las intervenciones que se han realizado sobre una location, en el desarrollo se ha cambiado el nombre de la pestaña a intervenciones”. En la figura 10.8 se muestra el prototipo del historial de intervenciones de location.

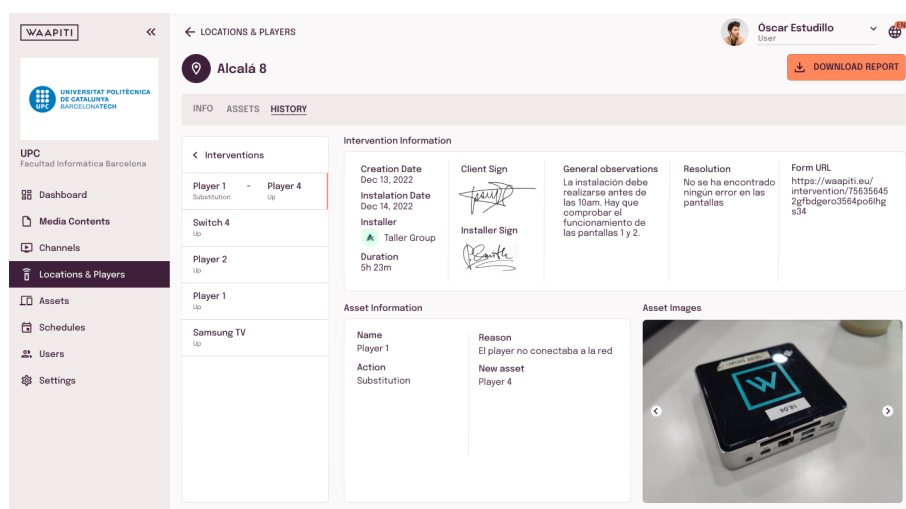


Figura 10.8: Historial de intervenciones de location

Fuente: Elaboración propia

Nueva intervención

La vista de nueva intervención permite crear una nueva intervención. En la figura 10.9 se muestra el prototipo de la nueva intervención.

WAAPITI

LOCATIONS & PLAYERS

Alcalá 8

INFO ASSETS HISTORY

< Spots

Player 1

Player 2

Samsung TV

Switch 4

No asset selected

+ NEW ASSET

1 Assets 2 Actions 3 Details 4 Review

Detail the action for each asset

Player 2

Action

Replace

Replacement new asset

Name

Player 3

Serial Number

Purchase Order

56341345453

Type

PLAYER

Model

NUC 3

Shop Date

25/03/2023

Warranty Period

2 years

Manufacturer

INTEL

Maintenance

Other details

Observations

Ubicar el player en la misma posición que el reemplazado.

BACK NEXT

Figura 10.9: Nueva intervención

Fuente: Elaboración propia

WAAPITI

LOCATIONS & PLAYERS

Alcalá 8

CREATE INTERVENTION

ADD 3D ID

VIEW MODE

NEW INTERVENTION

Player 1

UP

Samsung Replace

Switch 4

Issue

Player 1

Instalar junto a la pantalla.

Samsueng

Reemplazar por el mismo modelo ya instalado.

ADJUNTAR ARCHIVO

ADJUNTAR ARCHIVO

Installer

Taller Group

General observations

DISCARD

GENERATE FORM

+ NEW ASSET

Oscar Estudillo

User

EDIT INFO

REMOVE

Figura 10.10: Nueva intervención previo a valoración del equipo

Fuente: Elaboración propia

Capítulo 11

Implementación

En este capítulo se describe la implementación de todo el sistema desarrollado en este trabajo de fin de grado. Se explica con un nivel de detalle suficiente para entender el funcionamiento del sistema, pero sin profundizar tanto que dificulte el entendimiento de la lectura. Es por eso que algunas funcionalidades menores no se incluyen, aunque se puede ver su implementación de la API en el anexo.

El capítulo empieza por el desarrollo de backend y en las siguientes secciones se explican las diferentes funcionalidades e interfaces de la aplicación. La explicación se apoya en capturas de pantalla y otros elementos visuales para facilitar la comprensión. Todos aquellos elementos que aparezcan en imágenes pero no se incluyan en la explicación de ese apartado deben ser ignorados, pues se explicarán en el apartado correspondiente.

Debe quedar claro que en este proyecto el autor ha trabajado en todo el sistema, tanto backend como frontend.

11.1. Backend

Para poder entender correctamente el funcionamiento de la implementación de backend, es necesario entender los principios de NestJS y de TypeORM.

11.1.1. TypeORM

TypeORM es un ORM (Object Relational Mapping) que permite definir las entidades de la base de datos como clases de TypeScript. Esto permite que se puedan realizar consultas a la base de datos de forma sencilla y que se puedan definir relaciones entre las entidades.

NestJS tiene una integración muy sencilla con TypeORM, que permite simplificar una gran parte de la lógica de negocio.

Es necesario antes de poder utilizar TypeORM en NestJS, crear la configuración de la base de datos, que permitirá a las funciones de TypeORM realizar consultas reales. El módulo de TypeORM se importa a la aplicación principal de NestJS de forma que los servicios que este módulo incluye estén disponibles para todos los módulos dentro de la aplicación de NestJS.

Entidades

Las entidades son clases que representan las tablas de la base de datos. En la figura 11.1 se puede ver en dos imágenes parte de la clase de *Asset* como ejemplo.

En esta entidad, el decorador `@Column` indica que ese atributo se encuentra en la tabla de la base de datos. El decorador `@PrimaryGeneratedColumn` indica que ese atributo es la clave primaria de la tabla, y que se generará automáticamente.

En cuanto a las relaciones, los diferentes decoradores se corresponden al tipo de relación entre las diferentes entidades. TypeORM también permite asignar opciones a las relaciones como *cascade* que permite que al eliminar una entidad, se eliminen también las entidades relacionadas o *eager* que permite que al obtener una entidad, se obtengan también las entidades relacionadas automáticamente.

```
@Entity()
export class Asset {
  /** Id */
  @PrimaryGeneratedColumn({
    type: "int",
    name: "id",
  })
  id: number;

  /** Serial number */
  @Column("varchar", {
    nullable: true,
    name: "serialNumber",
  })
  serialNumber: string;

  /** Purchase order */
  @Column("varchar", {
    nullable: true,
    name: "purchaseOrder",
  })
  purchaseOrder: string;

  /** Name */
  @Column("varchar", {
    nullable: false,
    name: "name",
  })
  name: string;

  @OneToMany(type => AssetIntervened, assetIntervened => assetIntervened.asset, {
    assetsIntervened: AssetIntervened[];

    // TYPES
    @ManyToOne(type => AssetType, assetType => assetType.id, { eager: true, nullable
    @JoinColumn({ name: 'type' })
    type: AssetType;

    // Speaker
    @OneToOne(type => AssetSpeaker, assetSpeaker => assetSpeaker.asset, { eager: tru
    assetSpeaker: AssetSpeaker;

    // Amplifier
    @OneToOne(type => AssetAmplifier, assetAmplifier => assetAmplifier.asset, { eage
    assetAmplifier: AssetAmplifier;

    // Screen
    @OneToOne(type => AssetDisplay, assetDisplay => assetDisplay.asset, { eager: tru
    assetDisplay: AssetDisplay;

    // SOC
    @OneToOne(type => AssetSoc, assetSoc => assetSoc.asset, { eager: true, nullable:
    assetSoc: AssetSoc;

    // Player
    @OneToOne(type => AssetPlayer, assetPlayer => assetPlayer.asset, { eager: true,
    assetPlayer: AssetPlayer;

    // Replacement
    @OneToOne(type => AssetIntervenedReplace, asset => asset.assetIntervened, { null
    assetIntervenedReplace: AssetIntervenedReplace;
```

(a) Atributos

(b) Relaciones

Figura 11.1: Entidad de *Asset*

11.1.2. NestJS

NestJS combina elementos de OOP (Object Oriented Programming), FP (Functional Programming), and FRP (Functional Reactive Programming) para crear una API REST garantizando las mejores prácticas.

La estructura de NestJS se basa en módulos, controladores y proveedores.

Módulos

Los módulos son clases que permiten a NestJS organizar la aplicación. Estos módulos se deben importar en el módulo principal de la aplicación para que puedan ser utilizados, así como unos a otros para poder acceder a sus funcionalidades.

Por defecto, NestJS crea un módulo principal llamado *Application Module*.

Como ejemplo, en caso de tener un controlador de activos, y varios proveedores que se relacionen con la lógica de los activos, se encapsularán en un módulo llamado *Asset Module* que permitirá que al importar el módulo a otro (o al principal) se puedan utilizar todos los elementos que contiene.

En la figura 11.2 se puede ver un ejemplo de estructura de módulos.

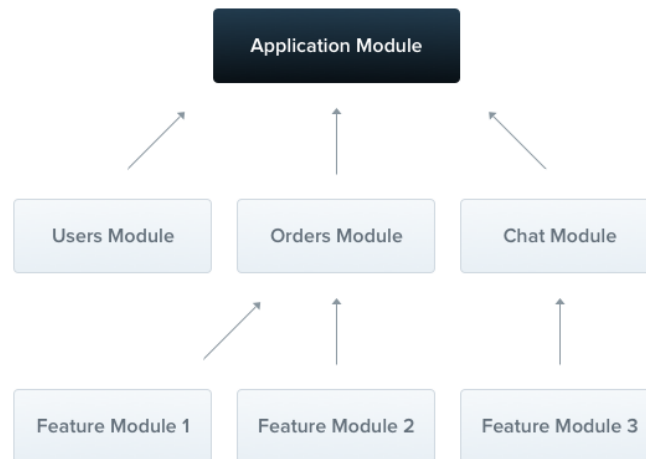


Figura 11.2: Estructura de módulos en NestJS

Fuente: NestJS [3]

Controladores

Los controladores son los encargados de recibir las peticiones HTTP y procesarlas. En una estructura simple de NestJS, los controladores realizan la lógica de negocio, sin embargo en este proyecto se ha decidido separar la lógica de negocio en los proveedores. Por tanto los controladores se encargan de recibir las peticiones HTTP y enviarlas a los proveedores.

En la figura 11.3 se puede ver un ejemplo de estructura de controladores.

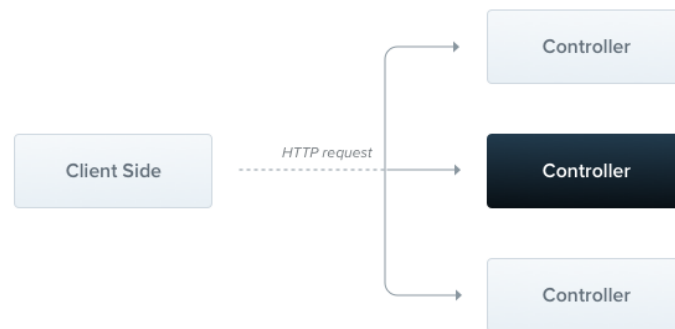


Figura 11.3: Estructura de controladores en NestJS

Fuente: NestJS [3]

Proveedores

Los proveedores son clases que pueden actuar como servicios, repositorios, factorías, helpers, etc. La idea de los providers en NestJS es que puedan ser inyectados como dependencias en los módulos o en otras clases para poder ser utilizados.

En este proyecto se han utilizado los proveedores como servicios, es decir, clases que contienen la lógica de negocio de la aplicación. Se han importado los servicios en los respectivos módulos para poder llamar al servicio desde los controladores correspondientes.

En la figura 11.4 se puede ver un ejemplo de estructura de proveedores.

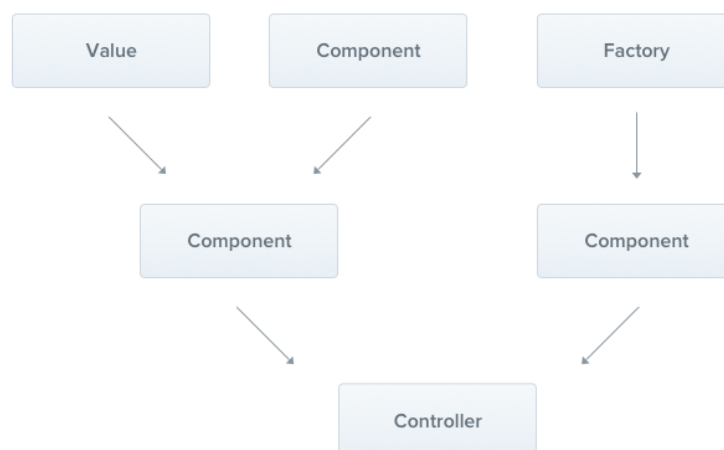


Figura 11.4: Estructura de proveedores en NestJS

Fuente: NestJS [3]

Estructura de ‘Asset’ en funcional

Para entender cómo se aplica NestJS en este proyecto, se va a utilizar como ejemplo la estructura de *Asset* en *functional*.

Business, a diferencia de funcional, no utiliza proveedores para todos los módulos, puesto que en la mayoría de casos el procesamiento de la petición es muy simple y se encarga el controlador directamente.

En la figura 11.5 aparece el decorador `@Module` que indica a NestJS que esa clase actúa como módulo. Para funcionar correctamente, se importan dos módulos de TypeORM que permiten acceder a los datos de la DB de *Cliente* y *LocationGroup* y se exporta el servicio de *Asset* para que pueda ser utilizado en otros módulos. Se define como controlador el controlador de *Asset* y se utilizan como providers las clases de *Asset* y *S3*.

Este módulo se añade al módulo principal de *functional* para que pueda ser utilizado.

```
@Module({
  imports: [
    | TypeOrmModule.forFeature([Client, LocationGroup]),
  ],
  controllers: [AssetsController],
  providers: [AssetsService, AWSS3Service],
  exports: [AssetsService],
})
```

Figura 11.5: Módulo de *Asset* en funcional

El controlador de *Asset* define todas las rutas de acceso a las que llama business¹ y se encarga de ejecutar las funciones del servicio importado. En la figura 11.6 se puede ver parte del código del controlador de *Asset*.

```
@Controller('assets')
export class AssetsController {

  constructor(private readonly service: AssetsService) { }

  @MessagePattern({ cmd: 'assets.list' })
  async getAssets(request: ListAssetsCoreRequest): Promise<CoreResponse> {
    | return this.service.listAssets(request.client, request.locationGroup,
  }

  @MessagePattern({ cmd: 'assets.find' })
  async getAsset(request: FindAssetCoreRequest): Promise<CoreResponse> {
    | return this.service.findAsset(request.id, request.user);
  }
}
```

Figura 11.6: Controlador de *Asset* en funcional

Finalmente el servicio de *Asset* es el encargado de realizar la lógica de negocio. En la figura 11.7 se puede ver el encabezado de la clase, el decorador `@Injectable` es el que

¹Como se explica en 8, funcional no acepta peticiones externas

define que la clase actuará como provider. También se puede observar como se inyectan proveedores de tipo repositorio, esto forma parte de TypeORM y permite acceder a las entidades de la base de datos.

```
@Injectable()
export class AssetsService {
  constructor(
    private readonly guard: GuardService,
    private readonly s3: AWSS3Service,
    @InjectRepository(Client) private readonly clientRepository: Repository<Client>,
    @InjectRepository(LocationGroup) private readonly locationGroupRepo: Repository<LocationGroup>,
  ) { }
```

Figura 11.7: Servicio de *Asset* en funcional

11.1.3. Ejemplo de lógica de negocio

La lógica de negocio es muy similar para todos los endpoints, por lo que se ha definido un esquema general para entender su funcionamiento. También se ejemplificará con el endpoint de *asset/:id*.

En la figura 11.8 se puede ver el esquema general de un endpoint. Resumidamente, business recibe la petición, y si es válida, la envía a funcional. En funcional se realizan las consultas y modificaciones necesarias en la base de datos y se devuelve la respuesta a business. Business se encarga de devolver la respuesta al cliente.

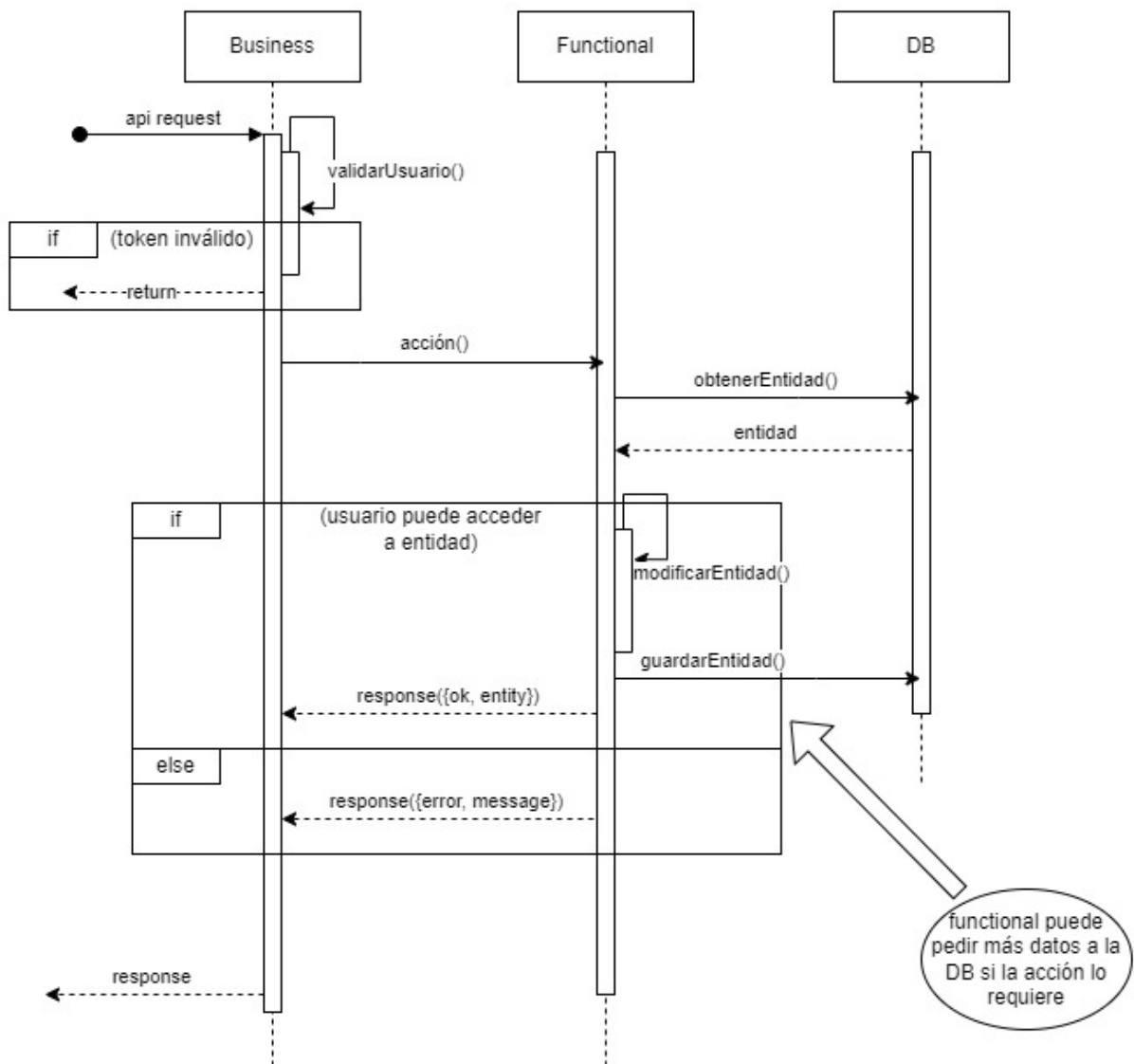


Figura 11.8: Esquema general de un endpoint

Fuente: Elaboración propia

Asset/:id

El primer paso para generar la lógica de obtener la información de un activo es definir la ruta de acceso en el servicio de *Business*. En la figura 11.9 se puede ver el código.

El decorador *@UseGuards* indica que se deben ejecutar una serie de funciones de validación previas al acceso a la función de obtener un activo.

Los otros decoradores permiten definir la ruta de acceso y garantizar la funcionalidad completa de TypeScript. Los decoradores *@ApiResponse* son parte de Swagger [47] y permiten generar documentación de la API.

Una vez se han procesado los datos de la petición, se envía la acción a *functional* para su procesamiento.

```

/**
 * Gets a specific asset
 * [GET] /asset/:id
 */
@Get('/:id')
@ApiBearerAuth("User Token")
@UseGuards(JwtGuard, UserGuard, PolicyGuard)
@ApiResponse({ status: HttpStatus.FOUND, description: 'Get asset by id successfully' })
@ApiResponse({ status: HttpStatus.NOT_FOUND, description: 'Asset not found' })
@UserPolicy([{ section: PolicySectionEnum.ASSETS, policy: AssetsPoliciesEnum.VIEW }])
async getAsset(
  @Req() req: AuthRequest,
  @Res() res: Response,
  @Param('id', ParseIntPipe) id: number,
): Promise<void> {
  const request: FindAssetCoreRequest = { id, user: req.user }
  const asset = await this.coreService.send<CoreResponse>('assets.find', request);
  res.status(asset.httpStatus).json(asset);
}

```

Figura 11.9: Ruta de acceso al endpoint de obtener un activo en business

En funcional, se accede a la función *findAsset* del servicio de *Asset*. En la figura 11.10 se puede ver el código.

Con ayuda de sintaxis de TypeORM (como alternativa a SQL), el primer paso es obtener el activo de la base de datos y en caso de que no exista o que no sea propiedad del cliente que ejecuta la petición se devuelve un error (líneas 172 y 174).

Seguidamente, para este caso particular es necesario obtener el enlace a las imágenes del activo (línea 176). Para ello se accede al servicio S3 de AWS y se obtiene el enlace de cada imagen y en caso de que no existan imágenes, se devuelve un array vacío.

Por último, se devuelve la información general del activo, la información específica del tipo de activo y las imágenes.

```

async findAsset(id: number, user: UserCache): Promise<CoreResponse> {
  try {

    const assetsQuery = getManager()
      .getRepository(Asset)
      .createQueryBuilder('asset')
      .leftJoinAndSelect('asset.type', 'type')
      .leftJoinAndSelect('asset.assetSpeaker', 'speaker')
      .leftJoinAndSelect('asset.assetAmplifier', 'amplifier')
      .leftJoinAndSelect('asset.assetDisplay', 'display')
      .leftJoinAndSelect('asset.assetPlayer', 'player')
      .leftJoinAndSelect('asset.assetSoc', 'soc')
      .leftJoinAndSelect('asset.assetsIntervened', 'ai')
      .leftJoinAndSelect('ai.images', 'images')
      .leftJoinAndSelect('ai.intervention', 'intervention')
      .where('asset.id = :id', { id })

    const asset = await assetsQuery.getOne();

    if (!asset) if (!asset) throw new AppError(ErrorTypeEnum.ASSET_NOT_FOUND);

    if (!(await this.guard.assetCheckOwn(asset, user))) throw new AppError(ErrorTypeEnum.ASSET_NOT_FOUND);

    if (asset.assetsIntervened?.length > 0) {
      for (let i = 0; i < asset.assetsIntervened.length; i++) {
        const assetIntervened = asset.assetsIntervened[i];
        if (assetIntervened.images?.length > 0) {
          for (let i = 0; i < assetIntervened.images.length; i++) {
            const image = assetIntervened.images[i];
            image.url = await this.s3.getFileByName("asset", image.name);
          }
          assetIntervened.images = assetIntervened.images.sort((a: any, b: any) => b.creationDate - a.creationDate);
        }
      }
    }

    return {
      httpStatus: HttpStatus.OK,
      asset
    };
  }
};

```

Figura 11.10: Llamada a la función de obtener un activo en funcional

11.1.4. Esquema de base de datos

Como se ha mencionado anteriormente, la base de datos principal del proyecto es MySQL. En la figura 11.11 se puede ver el esquema de la base de datos. Las relaciones indican las relaciones de clave foránea entre las diferentes tablas.

Para este proyecto, las clases de la base de datos se corresponden con las clases del modelo conceptual del dominio de la figura 7.1.

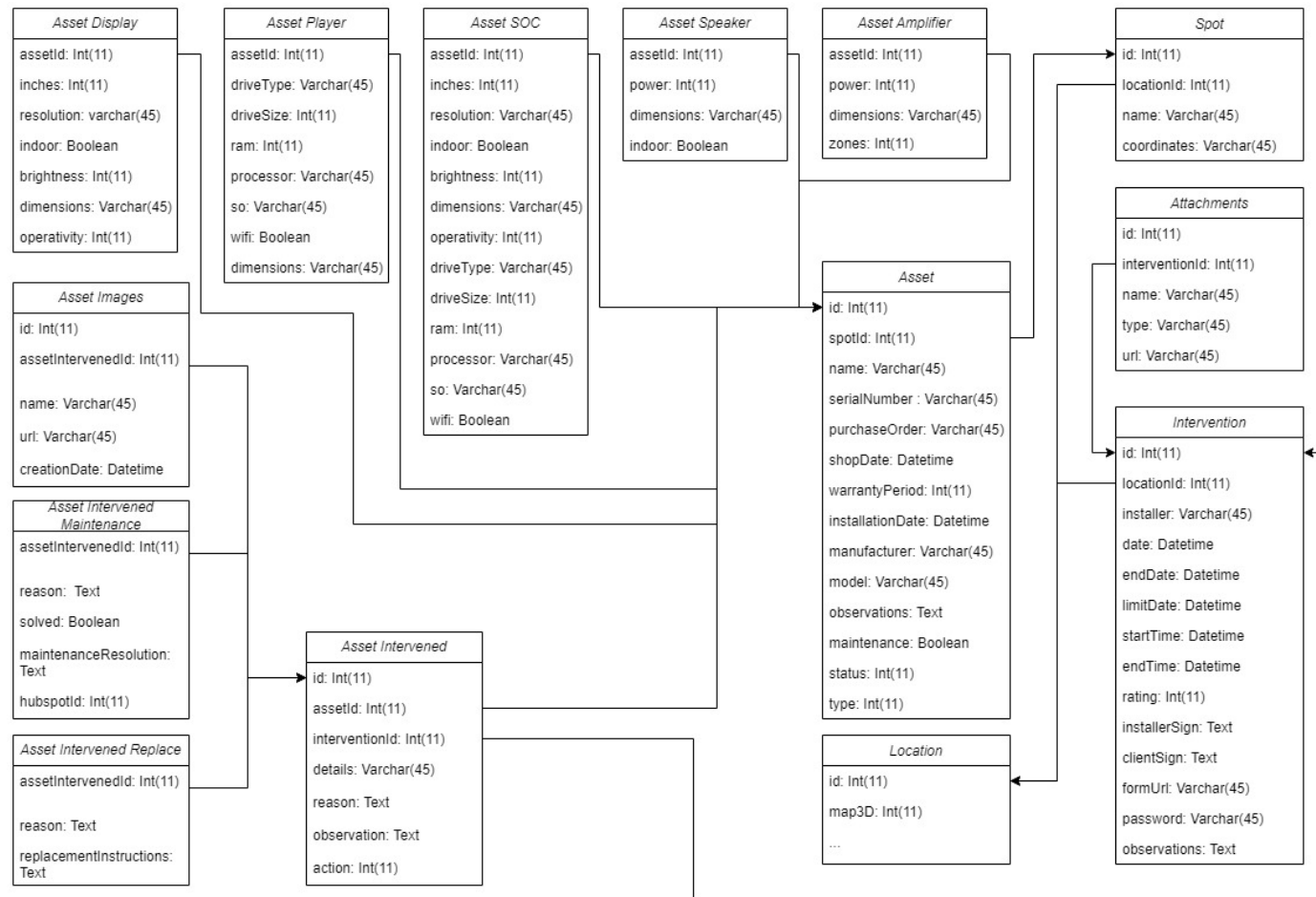


Figura 11.11: Esquema de la base de datos

Fuente: Elaboración propia

11.2. Frontend

Siguiendo el diseño de prototipos, se han desarrollado todas las interfaces necesarias para cumplir con todos los requisitos. En esta sección se explica el proceso de desarrollo de frontend.

11.2.1. Visualización de location / instalación

La primera interfaz que se debe entender es aquella en la que se pueden ver todos los activos que forman parte de una instalación en una location concreta.

Para empezar, es necesario entender que esta interfaz se encuentra dentro de la sección de una location en concreto. No es necesario entender la navegabilidad de la plataforma para llegar a seleccionar una location, pero sí saber que en la interfaz de una location antes solo existía información básica de esta, y ahora se ha incluido dos nuevos apartados. Estos nuevos apartados se han organizado en forma de *tabs* como se muestra en la figura 11.12.

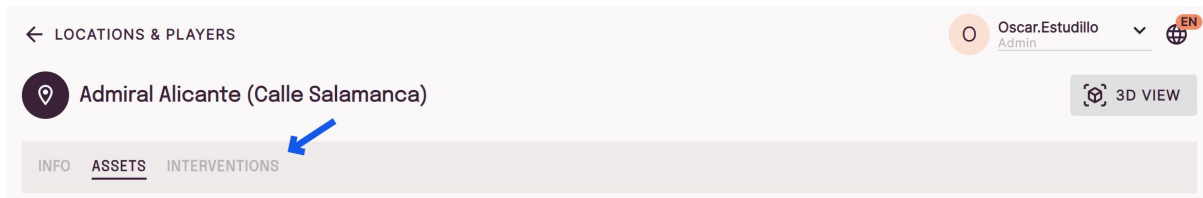


Figura 11.12: Tabs en interfaz de location

La funcionalidad descrita en esta sección se encuentra en el tab *Assets*.

Como se ha descrito anteriormente, los activos de una instalación se organizan en plantas y puntos ². Se ha creado un menú interactivo en el lateral izquierdo de la interfaz para poder navegar entre esta distribución y finalmente poder seleccionar un activo. Este menú se puede ver en la figura 11.13.

²En inglés *floors* y *spots*

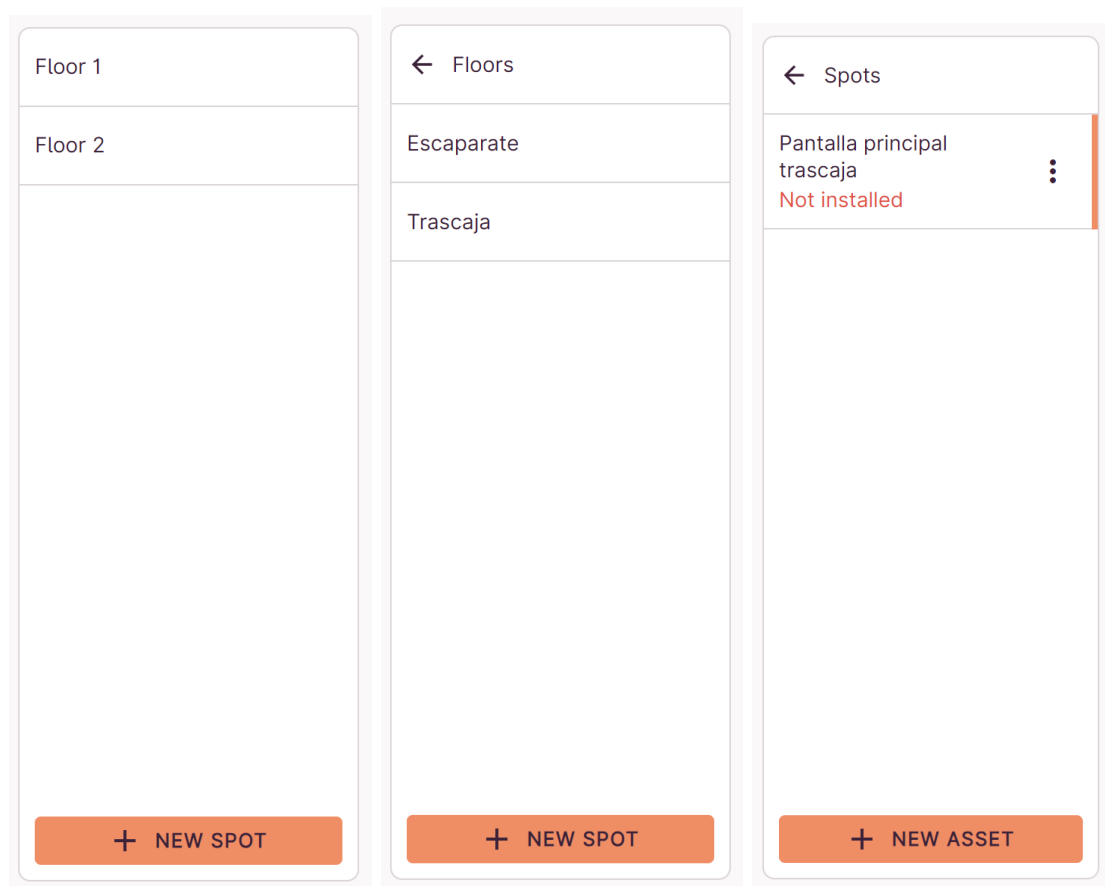


Figura 11.13: Menú de navegación de instalación

Para este punto solo existe un único activo, el cuál tiene como nombre *Pantalla principal trascaja*. Este aparece como seleccionado, lo que indica que la información que aparece en el apartado derecho de la interfaz es la de este activo en concreto. El activo en cuestión aparece como “No instalado”, lo cuál indica que no se ha realizado la primera intervención para este activo. Más adelante se explicará en detalle cómo se realiza esta primera intervención.

En la figura 11.14 se puede ver la interfaz al completo, con el menú de navegación a la izquierda y la información del activo seleccionado a la derecha. Los tabs y algunos elementos de la cabecera se encuentran en la parte superior.

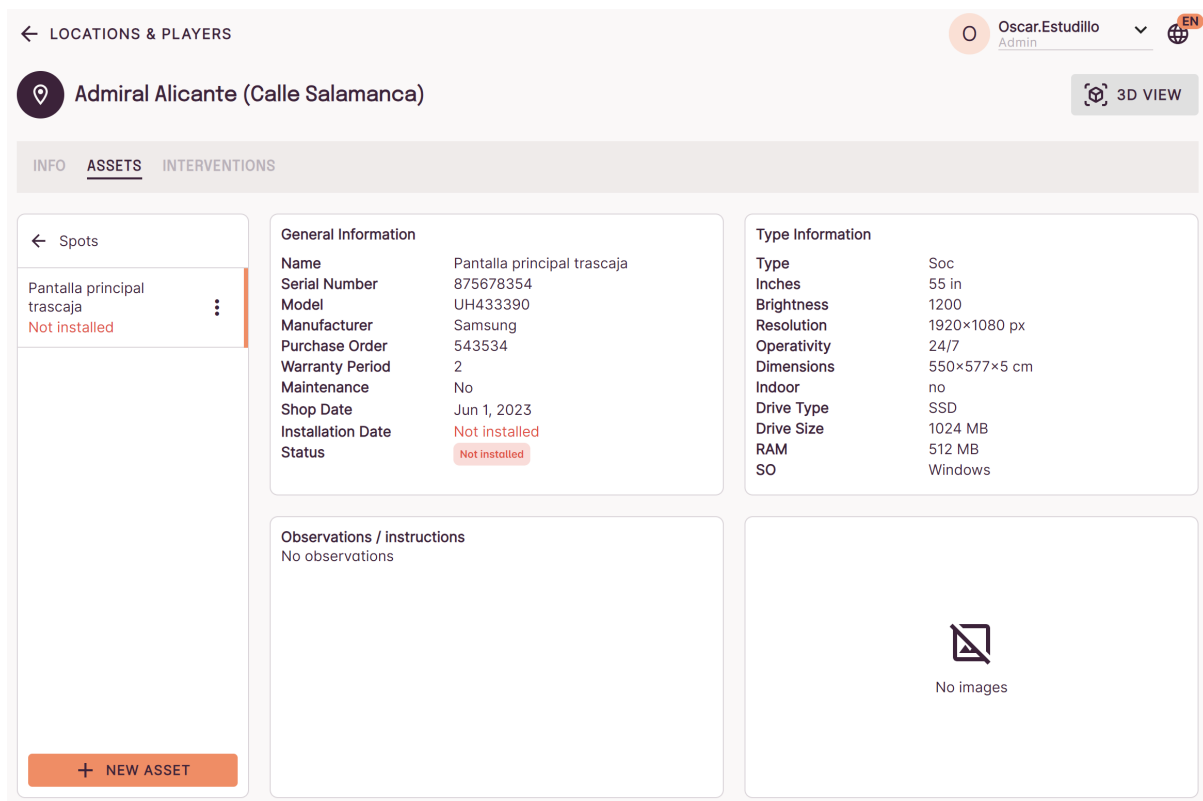


Figura 11.14: Interfaz de visualización de activos

Para esta interfaz se ejecutan dos llamadas a la api del proyecto. La primera obtiene información de la distribución de la location (en plantas y puntos) y la segunda se realiza para obtener toda la información del activo seleccionado. En las tablas 11.1 y 11.2 se puede ver el detalle de las llamadas a la api.

Descripción	Obtener los spots de una location
Método	GET
URL	/location/:id/spots
Parámetros	id: Number
Body	-
Respuestas	200: { spots: Object; httpStatus: Number; } 401: Unauthorized 404: Not Found

Tabla 11.1: API: Obtener spots de una location

Descripción	Obtener un activo
Método	GET
URL	/assets/:id
Parámetros	id: Number
Body	-
Respuestas	200: { asset: Object; httpStatus: Number; } 401: Unauthorized 404: Not Found

Tabla 11.2: API: Obtener información de un activo

11.2.2. Visualización de location / instalación con mapa 3D

Para facilitar la visualización de la información de una location, se ha desarrollado una interfaz que permite ver la distribución de la location en un mapa 3D.

El primer paso para visualizar el mapa 3D es crear este mapa en la página web que ofrece la librería utilizada (*SMPLRSpace*). En la figura 11.15 se puede ver el editor de mapas 3D.

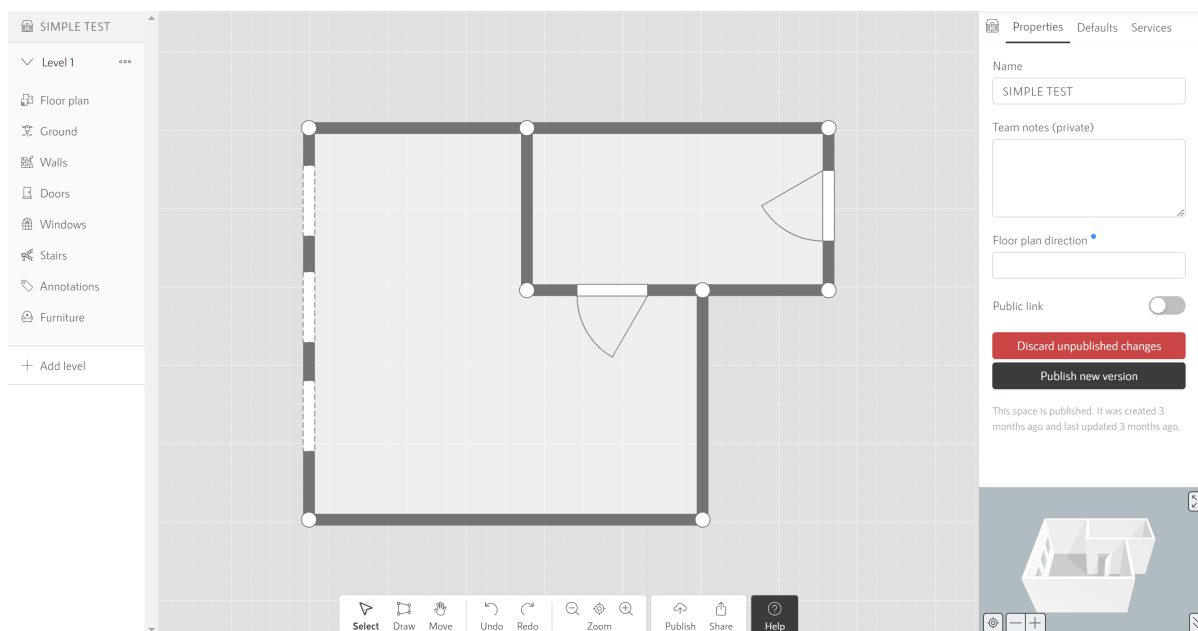


Figura 11.15: Editor de mapas 3D

Este mapa 3D está asociado a un ID que se debe introducir en la location. En la cabecera de la pestaña de activos se puede ver un botón para cambiar a la visualización 3D, el cuál solicitará el ID del mapa 3D si no ha sido ya introducido. En la figura 11.16 se observa esta funcionalidad.

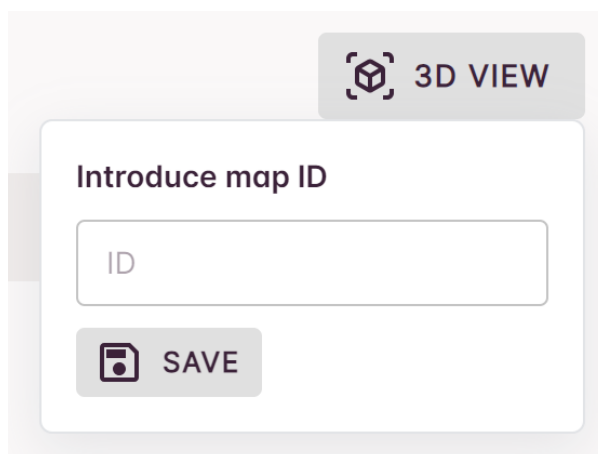


Figura 11.16: Solicitud de ID de mapa 3D

El uso de esta interfaz es similar a la vista anterior, con la única diferencia que los puntos están ubicados en el mapa 3D en vez de en una lista. En la figura 11.17 se puede ver la interfaz de visualización de location con mapa 3D.

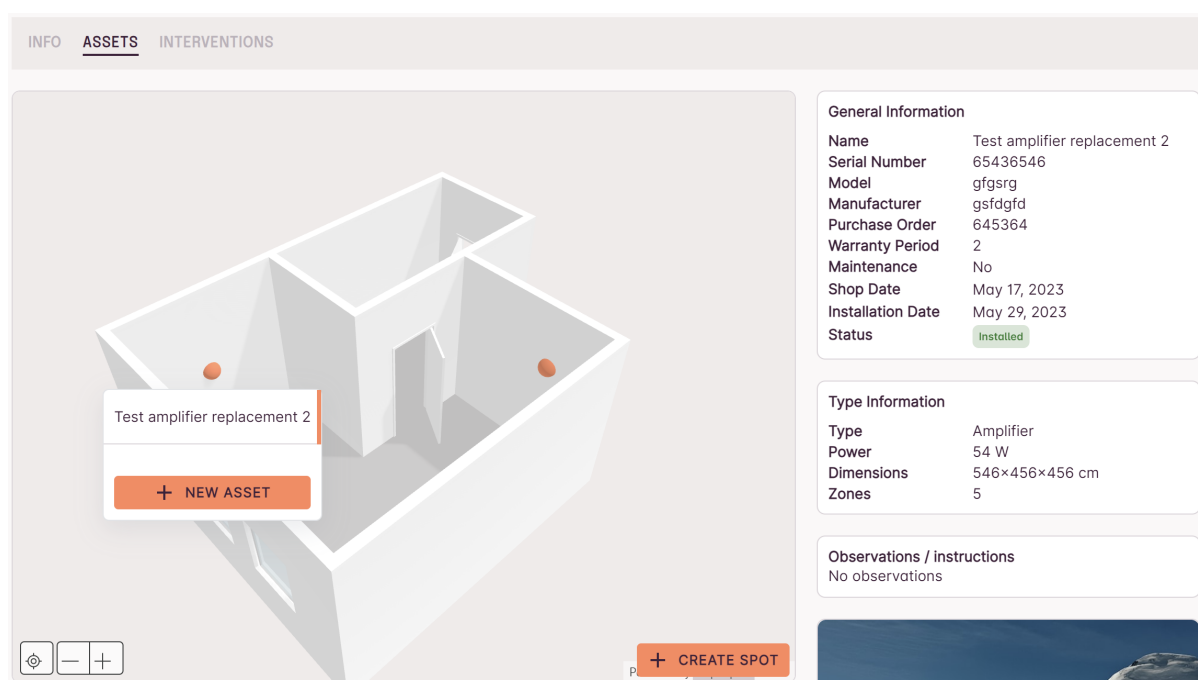


Figura 11.17: Visualización de location con mapa 3D

Las llamadas a la api para esta interfaz son las mismas que para la interfaz anterior, véase las tablas 11.1 y 11.2.

11.2.3. Creación de activo y punto

Para la creación de un activo o un punto se ha implementado un *drawer*³ que aparece haciendo click en el botón de “Crear activo” o “Crear spot” respectivamente. Estos botones se encuentran en el menú de visualización de activos o en el propio mapa 3D según la vista seleccionada, com se observa en las figuras 11.14 y 11.17.

Al hacer click desde el menú, se asigna automáticamente la planta (para el punto) y el punto (para el activo) a la información que se solicita para la creación de ambos. En la figura 11.18 se puede ver el formulario de creación de ambos elementos.

The image shows two side-by-side mobile application drawers. The left drawer is titled 'New Asset' and contains the following fields: a dropdown menu for 'Spot' with 'Trascaja' selected, a dropdown menu for 'Type', a text input for 'Name', a text input for 'Serial Number', a text input for 'Purchase Order', a text input for 'Model', a date picker for 'Shop Date', a numeric input for 'Warranty Period' set to '2' with 'years' as a unit, and a text input for 'Manufacturer'. The right drawer is titled 'New spot' and contains a text input for 'Name' and a dropdown menu for 'Floor' with 'Floor 1' selected. Both drawers have a close button (X) in the top right corner and 'CANCEL' and 'SAVE' buttons at the bottom.

Figura 11.18: Formulario de creación de activo y punto

En la tabla 11.3 se puede ver la información que se solicita para la creación de un activo y en la tabla 11.4 la información que se solicita para la creación de un punto.

³Menu lateral

Descripción	Crear un activo en un spot
Método	POST
URL	/assets
Parámetros	-
Body	spotId: Number name: String serialNumber: String purchaseOrder: String model: String shopDate: Date warrantyPeriod: Number manufacturer: String observations: String type: number
Respuestas	201: { asset: Object; httpStatus: Number; } 400: Bad request 401: Unauthorized

Tabla 11.3: API: Crear un activo

Descripción	Crear un spot en una location
Método	POST
URL	/spots
Parámetros	-
Body	name: Number locationId: Number floor: Number coordinates: Object description: String
Respuestas	201: { spot: Object; httpStatus: Number; } 400: Bad request 401: Unauthorized 404: Not Found

Tabla 11.4: API: Crear un spot

11.2.4. Visualizar y crear intervenciones

Para visualizar las intervenciones registradas en el sistema, se ha implementado una interfaz similar a la de visualizar activos. A esta interfaz se accede mediante el tab *Interventions*.

Para seleccionar la intervención, se ha creado un menú en la parte izquierda de la pantalla, donde en el primer paso se muestran todas las intervenciones y en el segundo todos los activos que se han intervenido para la intervención seleccionada. En la figura 11.19 se puede ver este menú.

El color rojo indica que la intervención no ha sido finalizada y su fecha límite de finalización ha pasado.



Figura 11.19: Menú de navegación de intervenciones

Tras seleccionar una intervención, se muestra la información de la misma en la parte derecha-superior de la pantalla. En la figura 11.20 se puede ver la interfaz al completo.

La información de cada activo intervenido aparece, junto a las imágenes correspondientes a la intervención, en la parte derecha-inferior de la pantalla.

The screenshot shows the 'INTERVENTIONS' section of a web application. On the left, there's a sidebar with a list of assets: 'Altavoz Principal' (Maintenance) and 'Amplificador zona ventas' (Install). The main area displays the details for the 'Altavoz Principal' intervention.

Intervention Information		Observations / general instructions	Form URL	Attachments	Client signature
Technician Pep	Creation date May 9, 2023	Te adjunto el fichero con la localización de los altavoces / amplificador	https://revamp.platform.waapi.ti.eu/interventions/installer?to ken=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiIsInR5cGE6bnRlc nZlbnRpb25JCI6NCwicGFzc 3dvcmlQIiI0b3Jzcn03bGlslmp 0aSI6bnVsbCwiaWF0IjoxNjgz NjMzMDc5fQ.OYYQ3Zz503IK S1eMV3JK5s78ml6OcL-14aRU		
Installation date May 9, 2023	Duration 0h 4m				

Asset information			
Name Altavoz Principal	Observations No observations	Action Maintenance	Action resolution Modified plan
Action resolution reason No se podia segun indicado foto	Reason Esta defectuoso	Solved No	

Below the asset information, there is a large image of a black speaker with three drivers.

At the bottom left, there is a button labeled 'CREATE INTERVENTION'.

Figura 11.20: Interfaz de visualización de intervenciones

Para crear una intervención se debe clicar el botón “Create intervention” que se puede ver en el menú de la figura 11.19. Al hacer click en este botón se abre un formulario similar al de creación de activos, pero con diferentes pasos que guiarán al usuario para completar la información de la intervención. En la figura 11.21 se puede ver este formulario.

The figure shows four sequential screens of a mobile application for creating an intervention, each with a progress indicator at the top (Assets, Actions, Details, Review).

- Assets:** 'Select assets to intervene'. It lists three assets: 'Altavoz Principal' (2345687), 'Pantalla principal trascaja' (875678354), and 'Amplificador zona ventas' (1111111111111111). Each asset has a plus icon to select it.
- Actions:** 'Provide an action for each asset'. It shows the first asset 'Pantalla principal trascaja' with a dropdown menu set to 'Install' and a text field for 'Instructions'.
- Details:** 'General details and attachments'. It includes a toggle for 'Auto Closable', a 'Limit Date' field set to '06/16/2023', and a text area for 'Observations / general instructions' with the note 'The intervention has to be finished before 10am.' There is also an 'ADD FILES' button.
- Review:** 'Review intervention'. It summarizes the intervention with 'OBSERVATIONS' (The intervention has to be finished before 10am.), 'LIMIT DATE' (16 June 2023), 'ATTACHMENTS' (a small image icon), and 'ASSETS' (listing the selected assets and their actions).

Each screen has 'BACK' and 'NEXT' buttons at the bottom, and the final 'Review' screen has a '+ CREATE' button.

Figura 11.21: Formulario de creación de intervención

Para poder realizar estas 3 acciones (ver intervenciones, información de intervención y crear intervención) se realizan tres llamadas a la api que se pueden ver en las tablas 11.5, 11.6 y 11.7 respectivamente.

Tras completar la creación de la intervención, automáticamente se copiará el enlace que deberá ser enviado al técnico que vaya a realizar la intervención para que pueda acceder a la información necesaria y completarla. Más detalles en la siguiente sección.

Descripción	Obtener las intervenciones de una location
Método	GET
URL	/location/:id/interventions
Parámetros	id: Number
Body	-
Respuestas	200: { interventions: Array; httpStatus: Number; } 401: Unauthorized 404: Not Found

Tabla 11.5: API: Obtener intervenciones de una location

Descripción	Obtener la información de intervención mediante id
Método	GET
URL	/interventions/installer
Parámetros	id: Number
Body	-
Respuestas	200: { intervention: Object; httpStatus: Number; } 401: Unauthorized 404: Not Found

Tabla 11.6: API: Obtener información de intervención

Descripción	Crear una intervención en una location
Método	POST
URL	/location/:id/interventions
Parámetros	id: Number
Body	assets: Array<AssetIntervened> observations: String autoClosable: Boolean limitDate: Date
Respuestas	201: { intervention: Object; httpStatus: Number; } 400: Bad request 401: Unauthorized 404: Not Found

Tabla 11.7: API: Crear intervención

11.2.5. Formulario de técnico instalador

Una de las claves de este proyecto es la generación automática del formulario que permite al técnico instalador ver y completar los datos de las intervenciones. Como se ha mencionado anteriormente, al crear una intervención se genera un enlace que permite al técnico acceder a dicho formulario.

Este formulario está pensado para visualizar en el móvil y busca la mayor sencillez para garantizar que el técnico pueda completar los datos de la intervención de forma rápida y sencilla. En la figura 11.22 se puede ver el formulario de técnico instalador. En esta intervención solo se ha incluido un activo para intervenir, pero en caso de que hubiera más, se mostrarían todos los activos de la intervención.

Destacar que al tratarse de un acceso público a la plataforma, se ha ideado un sistema de token y contraseña, que garantiza que con el enlace únicamente se puede acceder a la información de la intervención correspondiente. Una vez completada la intervención, el enlace deja de ser válido.

Admiral - Alicante (Calle Salamanca)

Address:
Admiral - Alicante (Calle Salamanca)

Installer Name: *

Observations / general instructions:
The intervention has to be finished before 10am.

Attachments:

START INTERVENTION

Assets: 1 (Assets), 2 (Checks), 3 (Review)

These are the assets to intervene, press next to see the details.

Pantalla principal traseja

Install

Pantalla principal traseja

Action
Install

Instructions
Install vertically

Details *

Select an option

Images * (Upload at least 1)

+ ADD IMAGES

Checks (Can be checked later)

display_1 ☐

display_2 ☐

Attachments

BACK **NEXT**

Assets: 1 (Assets), 2 (Checks), 3 (Review)

Make sure you have checked all the following:

Pantalla principal traseja

display_1 ☐

display_2 ☐

Attachments

BACK **NEXT**

Assets: 1 (Assets), 2 (Checks), 3 (Review)

Start and end time: *

Start Time: 09:06 End Time: 09:24

Technician signature: *

CLEAN **CONFIRM** ☒

Attachments

BACK **CLOSE INTERVENTION**

Assets: 1 (Assets), 2 (Checks), 3 (Review)

THIS INFORMATION HAS TO BE FILLED BY THE CLIENT

Rate the attendance: *

★★★★★

Client signature: *

CLEAN **CONFIRM** ☒

Finish the intervention before giving the phone back to the technician.

Attachments

FINISH INTERVENTION

Figura 11.22: Formulario de técnico instalador

En caso de que el enlace se intente modificar, el backend lo detectará como inválido en el servicio de *business* y no permitirá el acceso a información. Por otro lado, si la intervención ya ha sido completada, también se mostrará una pantalla donde no se puede acceder a más información. En la figura 11.23 se puede ver tanto el error que se muestra

en caso de que el enlace no sea válido como la interfaz que se muestra en caso de que la intervención ya haya sido completada.

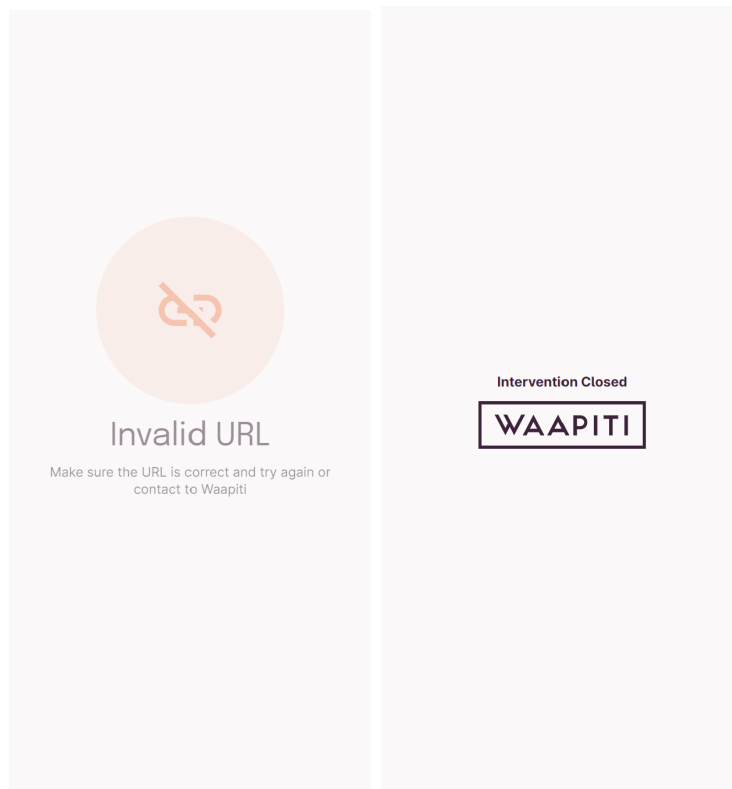


Figura 11.23: Error de acceso al formulario

Para evitar errores de conexión, se ha reducido el número de llamadas a la API a dos. Una para obtener los datos y otra para completar la intervención. En las tablas 11.8 y 11.9 se puede ver el detalle de los endpoints correspondientes.

Descripción	Obtener la información de intervención mediante token
Método	GET
URL	/interventions/installer
Parámetros	-
Body	-
Authorization	Bearer token
Respuestas	200: { intervention: Object; httpStatus: Number; } 401: Unauthorized 404: Not Found

Tabla 11.8: API: Obtener información de intervención

Descripción	Editar la información de una intervención con token
Método	PUT
URL	/interventions/:id
Parámetros	id: Number
Body	-
Authorization	Bearer token
Respuestas	200: { intervention: Object; httpStatus: Number; } 401: Unauthorized 404: Not Found

Tabla 11.9: API: Completar intervención

11.2.6. Lista de activos

Por último, se ha implementado una interfaz que permite al usuario ver todos los assets que tiene en todas sus locations. Esta interfaz resulta útil combinada con una serie de filtros que permiten encontrar un determinado grupo de activos y ver su información.

Esta interfaz detecta qué tipo de usuario accede a la lista, y se muestran únicamente los activos que el usuario tiene permiso para ver. Por ejemplo, un usuario administrador puede ver todos los activos de todos los clientes, mientras que un usuario de un cliente en concreto, solo verá los activos de su cliente.

En la figura 11.24 se puede ver un ejemplo de esta interfaz.

Assets					
Search Assets					
NAME	STATUS	TYPE	LOCATION ↓	WARRANTY	SERIAL NUMBER
Alargo 1	Not installed	Other	Admiral Elche <i>Admiral</i>	2 years	
Switch	Installed	Other	Admiral Elche <i>Admiral</i>	2 years	5666534
Test 2	Replaced	Other	Admiral - Alicante (Calle Salamanca) <i>Admiral</i>	2 years	5346
Test intervention	Not installed	Other	Admiral - Alicante (Calle Salamanca) <i>Admiral</i>	2 years	3456534
Pantalla principal trascaja	Installed	Display	Admiral - Alicante (Calle Salamanca) <i>Admiral</i>	2 years	563565
Test new asset map	In intervention	Other	Admiral - Alicante (Calle Salamanca) <i>Admiral</i>	2 years	34645
Test new edit	Installed	Other	Admiral - Alicante (Calle Salamanca) <i>Admiral</i>	2 years	532453456
Test new	In intervention	Other	Admiral - Alicante (Calle Salamanca) <i>Admiral</i>	2 years	3456543
Test amplifier replacement 2	Installed	Amplifier	Admiral - Alicante (Calle Salamanca) <i>Admiral</i>	2 years	65436546
Rows per page: 25					
1 / 0					

Figura 11.24: Lista de assets

En el momento de entregar el proyecto, se han implementado filtros de tipo y estado del activo. Para activarlos se accede mediante un menú lateral como se puede ver en la figura 11.25.

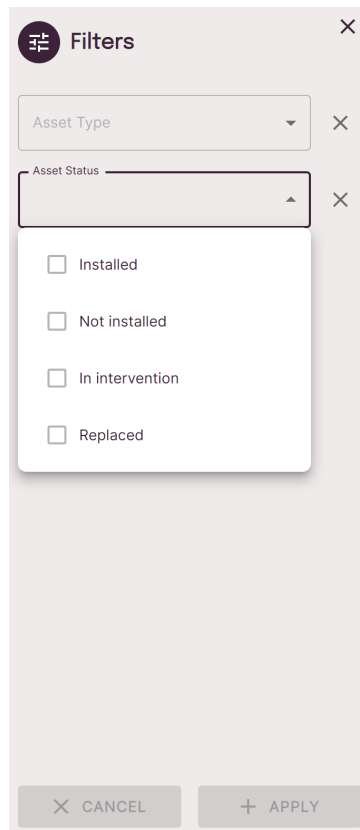


Figura 11.25: Filtros de la lista de assets

Finalmente, se ha incluido en esta vista una forma fácil de acceder a la información de un activo, así como las intervenciones en las que ha participado. A estas interfaces se accede mediante el icono de los tres puntos que se puede ver en la figura anterior. En la figura 11.26 se puede ver un ejemplo de la información de un activo y en la figura 11.27 se puede ver un ejemplo de las intervenciones. Ambas interfaces comparten diseño con las correspondientes vistas descritas en secciones anteriores.

En las figuras 11.26 y 11.27 se puede ver un ejemplo de la interfaz de información de un activo y de las intervenciones en las que ha participado.

INFO

INTERVENTIONS

General Information

Name

Pantalla principal trascaja

Serial Number

563565

Model

U8K22

Manufacturer

Samsung

Purchase Order

7356346

Warranty Period

2

Maintenance

No

Shop Date

Jun 5, 2023

Installation Date

Jun 8, 2023

Status

Installed

Type Information

Type

Display

Inches

55 in

Brightness

1200

Resolution

1920×1080 px

Operativity

24/7

Dimensions

100×40×5 cm

Indoor

yes

Observations / instructions

No observations

<



>

08/06/2023

Figura 11.26: Información de un activo

INFO

INTERVENTIONS

Install

08/06/2023

Intervention Information

Instructions

Install vertically

Action

Install

Action resolution

Install as described

<



>

Figura 11.27: Intervenciones de un activo

Para obtener la lista de activos solo es necesaria una llamada a la API, para la información de activo e intervenciones, se utilizan las mismas llamadas de secciones anteriores. En la tabla 11.10 se puede ver la llamada a la API para obtener la lista de activos.

Descripción	Obtener todos los activos
Método	GET
URL	/assets
Parámetros	take: Number page: Number s: String filters: String order: String client: Number locationGroupId: Number
Body	-
Respuestas	200: { data: Array; httpStatus: Number; page: Number; recordsTotal: Number; totalPages: Number; } 401: Unauthorized 404: Not Found

Tabla 11.10: API: Obtener todos los activos

Capítulo 12

Tests

En este capítulo se especifican todos los tests que se han realizado durante el transcurso del desarrollo del proyecto para comprobar que el sistema funciona correctamente.

Se han realizado tests de diseño y tests de integración. Los tests de diseño se han realizado para comprobar que el diseño del sistema es correcto y que los usuarios son capaces de entender y de utilizar las interfaces y los tests de integración se han realizado para comprobar que el sistema funciona correctamente.

No se han realizado tests unitarios puesto que no forman parte del proceso de desarrollo de la empresa, debido a que los tests de integración hasta el día de hoy han sido suficientes para comprobar el correcto funcionamiento de la plataforma. Sin embargo, en un futuro sería conveniente integrar los test unitarios en el proceso de desarrollo para comprobar de forma automática que no se han introducido errores en el código en el momento de realizar cambios en el mismo.

12.1. Tests de diseño y usabilidad

Se han realizado tests de diseño en dos etapas del proyecto. El primer test se ha realizado en la fase de diseño de prototipos y el segundo test se ha realizado en la interfaz ya implementada. Estos han sido los resultados obtenidos:

12.1.1. Test de diseño de prototipos

Objetivo

Comprobar que los usuarios son capaces de entender las funcionalidades de cada pantalla sin necesidad de explicación.

Método

Se ha presentado el diseño de forma ordenada a 5 empleados que han participado de forma indirecta en la especificación de requisitos del proyecto y se les ha pedido que describan qué funcionalidades creen que tiene cada pantalla. Una vez han terminado de describir las funcionalidades de una pantalla, se ha preguntado si estas funcionalidades son las que esperaban encontrar.

Resultados

Todos los usuarios han sido capaces de describir las funcionalidades de cada pantalla sin necesidad de explicación. Sin embargo, han tenido algunas confusiones debido a los nombres escritos en algunos botones de la interfaz que no eran los más adecuados.

Por otro lado, 3 de los 5 usuarios han comentado que en algunas pantallas tantas acciones les abruman y que les gustaría un diseño más simple.

Conclusiones

- Se ha cambiado el nombre de “history” a “interventions”.
- Se ha cambiado el componente de creación de asset e intervención de tipo “Modal” a tipo “Drawer”.¹

12.1.2. Test de diseño y navegabilidad de la interfaz implementada

Objetivo

Comprobar que un usuario es capaz de completar una serie de tareas sin haber utilizado la aplicación anteriormente.

Método

Se ha definido una lista de tareas ordenadas que el usuario de test debe completar de forma satisfactoria sin necesidad de explicación. El usuario de test es un usuario habitual de la plataforma y será quien use principalmente la herramienta desarrollada.

La lista de tareas es la siguiente:

1. Crea un punto en la planta 2 de la location “Admiral Alicante”.
2. Crea un activo en el nuevo punto de tipo amplificador.
3. Inicia una intervención para instalar el nuevo activo y hacer un mantenimiento del otro activo ya existente en la instalación.
4. Hazte pasar por instalador y completa la intervención.
5. Ves a la lista de activos de administrador y visualiza el activo creado e intervenido.

Resultados

El usuario ha sido capaz de completar fácilmente todas las tareas. Ha tenido un fallo de navegabilidad en el momento de crear una nueva intervención, puesto que se ha dirigido a la pestaña de intervenciones mientras que el botón de crear intervención está ubicado en la pestaña de activos.

Conclusiones

Se ha movido el botón de crear intervención a la pestaña de intervenciones para que sea más intuitivo para el usuario. Este cambio también ha sido validado por el product owner.

¹Ver componentes en el capítulo 10 de diseño

12.2. Tests de integración

Con tal de asegurar que todas las funcionalidades desarrolladas funcionan correctamente, se ha definido una lista de tests de integración que se han ido realizando a medida que se desarrolla el código. Estos tests se han realizado de forma manual y validan a la vez tanto frontend como backend.

Acción	Crear y eliminar puntos
Descripción	Crear un nuevo punto en una planta existente, una planta nueva, y eliminar uno de los dos
Resultado	Éxito

Tabla 12.1: Test de integración 1

Acción	Crear, modificar y eliminar activos
Descripción	Crear un nuevo activo, modificarlo y eliminarlo. Tanto en la lista como en el mapa 3D
Resultado	Éxito

Tabla 12.2: Test de integración 2

Acción	Crear y eliminar intervenciones
Descripción	Crear una nueva intervención y eliminarla.
Resultado	Éxito

Tabla 12.3: Test de integración 3

Acción	Completar intervenciones con acciones de tipo instalación, mantenimiento y reemplazo
Descripción	Completar las diferentes intervenciones y comprobar el estado de los activos
Resultado inicial	Fallo: No se crea correctamente el registro de intervención en activos nuevos creados como reemplazo
Modificación	Para el caso de reemplazo se deben crear dos activos intervenidos, para el reemplazado y el reemplazo
Resultado tras modificación	Éxito

Tabla 12.4: Test de integración 4

Acción	Visualizar activos en el mapa 3D
Descripción	Comprobar que todo en el mapa 3D es clicable y navegable
Resultado inicial	Fallo: Tras clicar en crear en crear un nuevo spot, no se puede clicar en los spots ya creados
Modificación	Se ha creado una función para controlar correctamente el evento que espera el click del usuario
Resultado tras modificación	Éxito

Tabla 12.5: Test de integración 5

Acción	Visualizar activos en la lista de activos
Descripción	Ver los activos buscando por uno en concreto y usando los filtros disponibles
Resultado	Éxito

Tabla 12.6: Test de integración 6